

分类号 _____

学校代码 10674

UDC _____

密 级 _____

昆明理工大学

硕士学位论文
(专业学位)

基于 Z 阶曲线边框扩展和最大块分割的范围查
询算法的研究

Research on Range Query Algorithm Based on Z-order Curve
Bounding Box Expansion and Maximal Block Segmentation

研 究 生 姓 名 _____ 汪荣进 _____

指导教师姓名、职称 _____ 贾连印 教授 _____

校外导师姓名、职称 _____ 李兴毅 高级工程师 _____

学 科 专 业 _____ 电子信息 _____

研 究 方 向 _____ 计算机技术 _____

论文工作起止日期 _____ 2024 年 9 月~2026 年 6 月 _____

论 文 提 交 日 期 _____ 2026 年 6 月 _____

遵守学术行为规范承诺

本人已熟知并愿意自觉遵守《昆明理工大学研究生学术规范实施细则（试行）》的所有内容，承诺所提交的毕业和学位论文是终稿，不存在学术不端行为，且论文的纸质版与电子版内容完全一致。

独创性声明

本人声明所提交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得昆明理工大学或其他教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。本人完全意识到本声明的法律结果由本人承担。

关于论文使用授权的说明

本人完全了解昆明理工大学有关保留使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。（保密的论文在解密后应遵守此规定）

本学位论文属于（必须在以下相应方框内打“√”，否则一律按“非保密论文”处理）：

- 1、保密论文：☒ 本学位论文属于保密。
- 2、非保密论文：☐ 本学位论文属于内部论文，网上延后公开。
- ☐ 本学位论文不属于保密范围，适用本授权书。

是否同意授权以下单位（必须在以下相应方框内打“√”，否则一律按“同意授权”处理）：

☒ 同意授权 ☐ 不同意授权

将本人学位论文著作权中的数字化复制权、发行权、汇编权和信息网络传播权的专有使用权在全世界范围内授予中国学术期刊（光盘版）电子杂志社，并在《中国优秀博硕士学位论文全文数据库》和 CNKI 系列数据库中出版。

研究生本人签名：汪荣进

签字日期：2026 年 5 月 20 日

研究生导师签名：贾连印

签字日期：2026 年 5 月 20 日

摘 要

随着移动互联网和物联网技术的快速发展，空间数据呈现爆发式增长，如何在海量数据中实现高效的空间范围查询已成为数据库领域的研究热点。传统的索引结构面对海量的数据显得力不从心，学习型索引为我们提供了一个新的解决思路，通过拟合数据分布，提升查询性能。现有学习型索引在处理多维空间数据时，通常需要借助空间填充曲线将其降维为一维，然后再借助一维索引结构实现范围查询。然而，由于 Z 阶曲线的固有不足，这种方式引入大量不在查询区域内的数据，需要最后一一验证排除，严重增加了不必要的 I/O 开销和过滤代价。

针对 Z 阶曲线会引入大量无效数据增加过滤代价问题，本文围绕基于 Z 阶曲线与学习型索引的空间范围查询优化展开研究，旨在减少无效数据访问，提升查询效率。本文构建了 Z 阶曲线和 PGM 混合的索引结构—ZPI，ZPI 通过 Z 阶编码将二维空间点映射为一维 Z 值序列，并利用 PGM 索引实现数据位置的快速预测。基于 ZPI，提出了高效的 ZPI-RQ 查询算法。为了避免引入太多不在查询区域内的点，提出了一种最大块分割策略，通过递归将查询区域分割成尽可能大的完整标准 Z 阶块，有效压缩了候选数据范围。实验结果表明，ZPI 索引在索引构建时间和内存占用方面都具有优异的表现，ZPI-RQ 查询算法相较于经典的 R*-tree 查询效率最高可提升 4.2 倍，相较于同样基于 Z 阶曲线的算法 ZM 提升最高可达 3.9 倍。

针对最大块分割在边界区域产生大量细小 Z 值区间（即区间碎片化）的问题，进一步实现 ZPI-RQ⁺算法，引入边框扩展机制，通过将查询区域对齐至固定阶网格边界，显著减少了区间数量；同时，通过对完全包含块与边界块进行分类验证，在保证查询结果完备性的前提下，进一步降低了验证开销。实验结果表明，ZPI-RQ⁺算法相较于经典的 R*-tree 查询效率可最高提升 5.1 倍，相较于同样基于 Z 阶曲线的算法 ZM 提升最高可达 6.3 倍。

关键词：空间范围查询；学习型索引；Z 阶曲线；ZPI；ZPI-RQ

Abstract

With the rapid advancement of mobile internet and the Internet of Things (IoT), spatial data has experienced explosive growth. Efficiently executing spatial range queries on such massive datasets has become a critical research focus in the database domain. Traditional indexing structures often prove inadequate when handling this data volume. Learned indexing offers a novel approach by modeling data distribution to enhance query performance. Current learned indexing methods for multi-dimensional spatial data typically employ space-filling curves for dimensionality reduction to one dimension, subsequently utilizing one-dimensional index structures for range queries. However, due to inherent limitations of the Z-order curve, this method retrieves substantial data outside the query region, necessitating post-filtering and significantly increasing unnecessary I/O operations and filtering costs.

To mitigate the excessive invalid data introduction and high filtering overhead caused by the Z-order curve, this thesis investigates spatial range query optimization based on the Z-order curve and learned indexing, aiming to curtail invalid data access and boost query efficiency. We construct a hybrid index structure, ZPI, integrating the Z-order curve with the PGM index. ZPI maps two-dimensional spatial points to a one-dimensional Z-value sequence via Z-order encoding and employs the PGM index for rapid data position prediction. Leveraging ZPI, we propose an efficient query algorithm, ZPI-RQ. To minimize the retrieval of points outside the query region, we introduce a maximal block segmentation strategy. This recursively partitions the query region into the largest possible complete, standard Z-order blocks, effectively narrowing the candidate data range. Experimental results demonstrate ZPI's superior performance in index construction time and memory footprint. The ZPI-RQ algorithm achieves up to 4.2 times higher query efficiency compared to the classic R*-tree and up to 3.9 times higher efficiency compared to ZM, another algorithm based on the Z-order curve.

Addressing the issue of numerous small Z-value intervals (i.e., interval fragmentation) generated by maximal block segmentation near boundaries, we

further develop the ZPI-RQ^+ algorithm. It introduces a bounding box expansion mechanism, aligning the query region to fixed-order grid boundaries to significantly reduce the number of intervals. Furthermore, by implementing classified validation for fully covered blocks and boundary blocks, it reduces verification overhead while ensuring the completeness of query results. Experimental results indicate that the ZPI-RQ^+ algorithm achieves up to 5.1 times higher query efficiency than the classic R^* -tree and up to 6.3 times higher efficiency than the ZM algorithm.

Keywords: Spatial range query; Learned index; Z-order curve; ZPI; ZPI-RQ

目录

摘 要.....	I
Abstract.....	III
1 绪论.....	1
1.1 研究背景及意义.....	1
1.1.1 研究背景.....	1
1.1.2 研究意义.....	2
1.2 国内外研究现状.....	2
1.2.1 基于树的索引.....	2
1.2.2 基于网格的索引.....	4
1.2.3 基于空间填充曲线的索引.....	5
1.2.4 学习型索引.....	6
1.3 论文出发点和主要工作.....	7
1.4 论文结构.....	8
1.5 本章小结.....	9
2 相关概念.....	11
2.1 基本定义.....	11
2.1.1 空间数据定义.....	11
2.1.2 空间范围查询.....	11
2.1.3 符号定义.....	12
2.2 Z 阶曲线.....	12
2.3 传统空间索引方法.....	13
2.3.1 R-tree 及其变种.....	13
2.3.2 四叉树与 k-d 树.....	14
2.3.3 网格类索引方法.....	15
2.4 学习型索引.....	15
2.4.1 学习型索引基本思想.....	15
2.4.2 PGM 索引.....	16
2.5 本章小结.....	18
3 ZPI 索引的设计与实现.....	19
3.1 引言.....	19

3.2 ZPI 索引结构.....	19
3.2.1 索引组成与构建流程.....	20
3.2.2 Z 阶曲线编码	22
3.3 索引构建.....	22
3.4 复杂度分析.....	22
3.5 实验分析.....	23
3.5.1 数据集及查询.....	23
3.5.2 实验设置.....	24
3.5.3 索引构建时间分析.....	25
3.5.4 索引空间分析.....	26
3.6 本章小节.....	27
4 基于 Z 阶曲线最大块分割的空间范围查询算法.....	29
4.1 引言.....	29
4.2 最大块分割算法设计动机	30
4.2.1 ZM 索引的问题分析.....	30
4.2.2 普通分割策略的局限性.....	31
4.3 最大块分割算法详细设计	32
4.3.1 相关定义.....	32
4.3.2 分割策略.....	33
4.3.3 最大块分割算法伪代码.....	35
4.3.4 算法正确性证明.....	36
4.3.5 区间数量与查询框形状的关系.....	37
4.4 基于最大块分割的查询算法	38
4.4.1 算法流程.....	38
4.4.2 与现有方法的对比分析.....	39
4.4.3 复杂度分析.....	40
4.5 实验分析.....	40
4.5.1 实验准备.....	40
4.5.2 r 对查询性能的影响	41
4.5.3 和其他算法对比.....	42
4.6 本章小节.....	43

5 基于 Z 阶曲线边框扩展的最大块分割范围查询算法.....	45
5.1 引言.....	45
5.1.1 问题分析.....	45
5.1.2 研究动机.....	46
5.2 查询边框扩展机制.....	46
5.2.1 查询边框扩展.....	46
5.2.2 扩展参数 m 的选择	48
5.3 基于边框扩展的最大块分割	48
5.3.1 改进的分割策略.....	48
5.3.2 算法正确性证明.....	50
5.3.3 QuadSplit ⁺ 算法伪代码	51
5.4 ZPI-RQ ⁺ 查询算法	53
5.4.1 算法设计思想.....	53
5.4.2 ZPI-RQ ⁺ 算法伪代码	53
5.4.3 复杂度分析.....	54
5.5 实验分析.....	55
5.5.1 实验准备.....	55
5.5.2 m 对查询性能的影响.....	55
5.5.3 r 对查询性能的影响	56
5.5.4 空间填充曲线对查询性能的影响.....	57
5.5.5 与 ZPI-RQ 的对比.....	58
5.5.6 和其它算法对比.....	59
5.6 本章小结.....	60
6 结论.....	63
6.1 本文工作总结.....	63
6.2 工作展望.....	63
致谢.....	65
参考文献.....	67
附录 A 攻读硕士学位期间的学术成果.....	73
附录 B 攻读硕士学位期间参与的科研项目.....	75

1 绪论

1.1 研究背景及意义

1.1.1 研究背景

随着移动互联网的普及与物联网、智慧城市等新兴业态的落地，空间数据的规模经历爆发式增长。从日常导航定位、即时配送调度，到社交平台的位置签到，几乎所有线下服务场景都与地理信息深度耦合^{[1][2]}。Foursquare、百度地图、微信等平台每日产生的空间数据记录已达海量级别，不仅体量庞大，且更新频繁，这对底层数据管理系统的查询能力提出了严峻挑战^{[3][4]}。如何在万亿级数据中实现快速、精准的空间检索，已成为数据库领域亟待突破的核心问题^[5]。

范围查询是各种空间操作中最基本的一类操作，也是应用最广泛的一类。其核心是快速筛选出所有落入该区域的数据点，在给定矩形区域内（或其他简单几何形状）^[6]。无论是浏览地图时的兴趣点加载，还是基于位置的个性化推荐，抑或是区域统计和分析都高度依赖高效的范围查询支持。但区间查询的执行成本却随着数据规模的不断扩大而大幅上升。如果底层指标结构设计不当，将会导致直接影响用户体验的查询延迟明显增多^[7]。

以 R 树及其改进版 R*树为代表的传统空间指数长期占据优势地位^[8]。这类索引在千万级别以下的数据集中表现良好，通过层次化树结构来组织空间数据^[9]。然而，树结构层级加深、节点频繁分裂重构等问题在数据量攀升到几亿级别的时候逐渐暴露出来，造成了构建和查询成本的大幅提升^{[10][11]}。传统索引方法的局限性在大尺寸、高并发的情景下日益突出。

作为一种颠覆性的设计理念，学习型索引近年来已经崭露头角。与传统索引依赖固定数据结构不同，学习型索引利用累积分布函数对数据的存储位置进行近似预测，通过模型拟合数据分布规律^[12]。这种方法可以有效压缩查找路径，减少访问次数，在一维数据上效果显著。但多维空间数据缺乏直接套用已有模型的一维次序。因此，当前研究的关键突破口是如何实现多维数据到一维空间的有效映射，并且保持较高的查询效率。空间填充曲线为此提供了一种解决办法。以 Z 阶曲线为例，它将二维坐标通过递归空间划分和编码的方式转化为一维 Z 值序列，并且可以将空间的邻接特性保留到一定程度。基于这个映射，学

习索引可以构建在 Z 值序列上,从而间接支持多维查询。但在实际应用中发现,虽然实现了简单的 Z 阶曲线,却由于 Z 阶曲线的跳变性会将大量无关数据点(无效点)混杂在查询区域所对应的 Z 值范围内。查询后需要对这些无效点进行逐一过滤,当数据规模较大时,这部分开销会明显影响整体效率。因此,以 Z 阶曲线为基础的学习索引性能提升的核心课题就是如何有效抑制无效点访问。

1.1.2 研究意义

本文以上述背景为基础,以空间范围查询优化为主线,在 Z 阶曲线与学习型索引的融合框架下展开研究。研究定位不是颠覆性创新,而是在现有结构基础上进行针对性改进,核心目标是将无效点访问和由此产生的额外计算支出最小化,前提是确保正确的查询结果。同时,在方案设计上兼顾结构简洁性和工程可行性,避免了引入复杂模型或维护费用过高的情况发生。

从实际应用层面来看,提升范围查询效率具有直接的应用价值。在位置服务系统中,查询响应时间直接影响用户的体验,更快的查询意味着更流畅的交互,而延迟则直接拉低用户体验的满意度。特别是在大规模数据场景下,即使是单次查询的微小优化,经过大规模海量请求的累积,也能转化为显著的系统资源节约。因此,查询过程的优化不仅能够改善服务质量,还能有效降低系统的整体运营成本。

从研究层面来看,学习型索引在多维空间数据上的应用仍然还在发展阶段。通过将 Z 阶曲线与学习型索引有机结合,并针对范围查询中的无效点问题进行改进,可为多维学习型索引的设计提供新的思路。本文提出的最大块分割与边框扩展策略,试图在减少无效点访问与控制区间数量之间寻求平衡。这一思路不仅提升了当前方法的查询效率,也为后续在更高维数据、动态更新场景下的研究提供了参考方向。

总体来看,本文分析现有方法在范围查询过程中的不足并提出了解决方案,对基于 Z 阶曲线学习型索引进行了优化。通过理论分析和实验验证相结合的方式,提出了一种提高大规模空间范围查询效率的可行路径,可以说具有实践价值和研究意义。

1.2 国内外研究现状

1.2.1 基于树的索引

空间范围查询在空间数据库里属于最基础也最经常被用到的操作之一，关于它的理论探讨和技术摸索能够一直追溯到上世纪八十年代初期。在那时，随着地理信息系统和计算机辅助设计应用的兴起，如何高效地组织和检索多维空间数据成为数据库领域最需解决问题，早期的研究工作主要围绕如何构建专门针对空间数据特征的索引结构展开，试图突破传统一维索引（如 B 树）在处理多维数据时的局限性。在这一时期，以树型结构为代表的空间索引方法逐渐成为研究焦点，为后续数十年的技术演进奠定了理论基础。

R-tree 及其改进版本^[8]是基于树的索引中最具影响力的成果之一。该结构由 Guttman 于 1984 年提出，它的核心设计思想是利用最小外接矩形（Minimum Bounding Rectangle，简称 MBR）来逼近和描述复杂的空间对象。在 R-tree 中，每个树节点都对应一个磁盘页面，内部节点里面存储其子节点的 MBR 及指针，叶节点则存储实际空间对象的 MBR 及其标识符。这种层次化的组织方式使得 R-tree 能够像 B 树一样保持平衡，而且还支持数据的动态插入与删除操作。在执行空间范围查询时，算法从根节点出发，递归地判断当前节点的 MBR 是否与给定的查询区域存在交集：如果存在交集，则继续向下访问相应的子节点；如果不存在交集，则直接剪枝，避免无效的磁盘访问^[13]。这种基于 MBR 相交判断的剪枝策略，有效减少了查询过程中需要访问的节点数量，从而提升了查询的效率^[14]。

在 R-tree 的基础上，研究人员针对其节点分裂策略进行了持续优化，其中最具有代表性的改进当属 R*-tree^[11]。R*-tree 通过引入更复杂的节点分裂准则，综合考量了节点覆盖面积、节点间重叠区域以及节点周长等多个要素，旨在最小化节点 MBR 之间的重叠程度和覆盖范围。这一改进显著减少了查询路径上需要遍历的分支数量，尤其是在数据分布较为密集的场景下，R*-tree 的查询性能较原始 R-tree 有了明显提升。正因如此，R-tree 被广泛应用于 PostgreSQL、Oracle Spatial 等主流数据库系统和地理信息软件中，成为事实上的工业标准之一。

除了 R*-tree 系列外，四叉树也是一种被广泛研究的树型空间索引结构^[15]。四叉树的基本原理是把二维空间递归地划分成四个大小相等的子区域，直到每个子区域包含的数据点数量低于预设阈值或者达到最大递归深度^[16]，这种划分方式和金字塔式的空间分辨率层次相吻合，使得四叉树特别适用于地图缩放、图像压缩以及点数据的快速定位等场景^[17]，四叉树的优势在于结构直观、实现简单，且完全在内存中运行时能够提供极快的访问速度^{[18][19]}。然而，其局限

性同样显而易见：当空间数据分布极不均匀时（例如城市中心区域数据密集而郊区稀疏），四叉树容易产生深度差异巨大的子树，导致树结构失衡，部分分支路径过长，从而影响查询效率的稳定性^[20]。

对于 R-tree 等树型结构而言，随着数据量的增长，树的高度不断增加，一次查询需要访问的节点路径变长，磁盘 I/O 次数随之上升。更为严重的是，频繁的数据插入和更新操作会引发节点分裂、合并与重平衡，这些维护操作在高并发场景下可能成为系统瓶颈。

1.2.2 基于网格的索引

与树型索引采用层次化组织的思路不同，网格索引（Grid Index）选择了一种更为直接的“空间驱动”策略。其核心思想是将目标空间按照某种规则划分为有限个网格单元，每个单元负责存储落入该区域内的空间对象^[22]。这种基于划分的索引方法在实现上极为简洁，构建速度快，且易于并行化处理^[23]。

均匀网格索引（Uniform Grid，简称 UG）是最简单的网格索引形式。它将空间划分为一系列大小完全相同的网格单元，每个单元通过二维数组或哈希表进行管理。当进行范围查询时，只需计算出查询区域覆盖了哪些网格单元，然后直接访问这些单元并过滤其中的数据点即可^[24]。UG 的优点在于构建时间复杂度仅为 $O(n)$ ，常数因子极小，且无需维护复杂的树结构。然而，其缺陷同样突出：在数据分布呈现高度倾斜时（如城市 POI 数据密集于市中心而稀疏于郊区），部分网格单元可能包含数以万计的数据点，而另一些网格则几乎为空。这种负载不均现象导致密集网格成为查询瓶颈，严重时甚至劣于顺序扫描^[25]。

针对均匀网格的上述问题，研究者提出了等深网格索引（Equal-Depth Grid，简称 EDG）。EDG 不再追求网格面积的均等，而是以网格内数据量均衡为目标，通过自适应调整网格边界，使每个网格单元包含大致相当的数据点。这种划分方式能够有效缓解数据倾斜带来的负载不均问题，在 Foursquare 签到数据、微博位置数据等具有明显热点区域的应用场景中表现出更优的查询性能。EDG 的代价在于构建过程需要统计数据的分布情况并动态确定划分边界，因此构建复杂度略高于 UG，但换来的查询性能提升在多数场景下是值得的。

对于四叉树而言，数据分布不均导致的树深度失衡问题在大规模数据下愈发明显，部分查询路径可能长达数十层，严重影响响应时间的稳定性。而对于网格索引，无论是 UG 还是 EDG，在面对复杂形状的查询区域或多维查询时，其性能往往难以保证，且难以扩展到三维或更高维度的空间数据。

1.2.3 基于空间填充曲线的索引

随着数据维度的增加，传统多维索引结构面临着一系列难以回避的挑战。**R-tree** 及其变种在高维空间中容易出现节点重叠率上升、剪枝效率下降的问题，而网格索引则面临网格数量爆炸式增长的困境。为了规避直接构建复杂多维结构所带来的开销，研究者提出了一种间接思路：将多维空间数据映射到一维空间，然后借助成熟的一维索引技术（如 **B+树**）进行管理和查询^[26]。在这一思路的指引下，空间填充曲线逐渐走进研究者的视野，成为连接多维数据与一维索引的重要桥梁。

在众多空间填充曲线中，**Z 阶曲线**（**Z-order Curve**）和 **Hilbert 曲线** 是最具代表性的两种。**Z 阶曲线** 通过将多维坐标的二进制表示按位交错组合，生成一个整数值作为编码。例如，对于二维坐标 (x, y) ，将其二进制位交错排列后得到的 **Z 值** 能够在一定程度上保留原始空间中的邻近关系^[30]。这种编码方式实现极为简单，仅涉及位运算，计算效率极高，因此被广泛应用于多维数据索引、图像压缩、数据布局优化等领域^{[28][29]}。

相比之下，**Hilbert 曲线** 在空间局部性保持方面表现更为出色^[31]。研究表明，**Hilbert 曲线** 能够更好地将空间邻近的点映射到一维序列中相邻的位置，从而减少查询过程中的误报率^[32]。然而，这种优越性是以编码和解码过程的复杂性为代价的。**Hilbert 曲线** 的转换涉及状态机、方向变换等复杂操作，计算开销远高于 **Z 阶曲线**^[33]。因此，在实际系统设计中，需要在局部性保持与计算效率之间做出权衡。

基于空间填充曲线的索引方法通常遵循两阶段的构建流程：首先，利用选定的空间填充曲线将多维空间数据（如二维坐标点）转换为一维编码值；随后，在这些编码值构成的一维序列上构建传统的索引结构，如 **B+树**、跳跃表，或是近年来备受关注的学习型索引（**Learned Index**）。这种先降维再用一维索引的组合策略巧妙地规避了直接构建多维索引的复杂性，同时能够复用一维索引领域数十年的研究成果。

以 **Z 阶曲线** 为例，每个空间点经过编码后得到一个 **Z 值**，所有 **Z 值** 按升序排列后形成有序序列。在此基础上构建的一维索引可以像处理普通一维数据一样执行点查询和范围查询。当需要进行空间范围查询时，系统首先将查询区域映射为 **Z 值** 空间中的若干区间，然后通过一维索引快速定位这些区间内的数据，最后通过坐标验证确保结果的准确性。

尽管上述方法在理论上具有可行性，但在实际应用于范围查询时，空间填充曲线暴露出了一个固有缺陷。对于一个给定的矩形查询区域，它在 Z 值空间中通常不会恰好对应一个连续的区间，而是可能被分割成多个不连续的片段。如果只是简单地用查询区域左上角和右下角的 Z 值来确定一个整体的 Z 值区间，则必然会将大量实际位于查询区域之外的点纳入该区间内。这些被误纳入的点虽然在 Z 值序列上落在查询区间内，但在原始空间坐标上并不满足查询条件，因此被称为无效点。无效点的存在带来了额外的计算开销。在索引定位到候选 Z 值区间后，系统必须遍历区间内的每一个点，逐一还原其空间坐标并与查询区域进行比较验证。当无效点比例较高时，这部分验证开销可能远超索引查找本身的时间，严重拖累整体查询性能。这一问题在查询区域形状不规则、边界与 Z 阶曲线走向不吻合时尤为突出。

1.2.4 学习型索引

2018 年，Kraska 等人提出了一种颠覆性的索引设计理念^[12]：利用机器学习模型替代传统的树结构索引，这一工作标志着学习型索引研究的正式兴起。其核心在于：传统索引（如 B 树）本质上可以看作是对数据分布位置的一种模型，只不过这种模型以树结构的形式存在。如果能通过机器学习算法直接学习数据的累积分布函数（CDF），就可以用训练好的函数模型来预测每个键值在存储结构中的位置，从而避免树结构带来的多次层级访问开销^[34]。这一思想一经提出，便迅速引发了学术界和工业界的广泛关注。

在一维数据场景中，学习型索引的研究取得了快速发展。Kraska 等人提出的 RMI 采用分层模型架构^[12]，通过多个模型的级联来逐步细化位置预测，在百万级至亿级数据规模下展现出了优于传统 B 树的查找性能。随后，研究者针对 RMI 存在的模型复杂、训练开销大等问题进行了改进：PGM-index^[36]和 FITing-tree^[35]等方法采用分段线性模型替代复杂的神经网络，通过贪心算法拟合数据分布，在保证误差范围可控的前提下，显著提升了模型构建效率和预测稳定性。为了支持动态数据更新，ALEX^[37]、LIPP 等^[38]结构进一步引入了缓冲区机制和增量更新策略，使学习型索引在面向实际生产环境中的频繁更新场景时具备了更强的可行性。

然而，上述方法主要针对一维数据设计，无法直接应用于多维空间数据。根本原因在于，多维空间缺乏天然的线性顺序，二维空间中的点无法像一维数值那样直接比较大小并排序。这一特性使得多维数据无法直接作为学习型索引

的输入。为了解决这一难题，研究者普遍采用先降维，再学习的技术路线：首先通过某种映射机制将多维坐标转换为一维编码值，然后在一维编码序列上构建学习型索引。

在这一思路的指引下，一系列多维学习型索引方法相继提出。ZM 索引^[39]是最早的尝试之一，它采用 Z 阶曲线将二维空间点映射为一维 Z 值序列，然后基于 RMI 模型构建学习模型，实现对 Z 值位置的快速预测。RSMI^[40]在 ZM 的基础上进一步优化，通过重新排列数据点的分布顺序，减小 Z 值序列中的间隔，从而提升模型的拟合效果和查询效率。MLI^[41]则借鉴了 iDistance 的思想，通过选取若干参考点将高维空间划分为不同区域，计算每个点到参考点的距离作为一维映射值，再构建学习型索引。LISA^[42]采用网格划分与测度映射相结合的方式，先对空间进行精细网格划分，再基于勒贝格测度将网格映射为一维值，最后构建学习模型。

尽管上述方法在点查询场景下表现出了优于传统索引的潜力，但在面向空间范围查询时，它们普遍面临一个共同的挑战。如前文所述，范围查询需要找出一个给定区域内的所有数据点。当查询区域通过降维映射转换为一维编码区间后，这个区间往往包含大量实际位于查询区域之外的无效点。这是因为空间填充曲线（如 Z 阶曲线）或距离映射（如 iDistance）只能在一定程度上保持空间邻近性，无法做到精确对应。

1.3 论文出发点和主要工作

从前文分析可以看出，传统空间索引结构在大规模数据和高频范围查询场景下容易出现访问开销大、维护成本高等问题；而基于空间填充曲线的降维方案虽然能够将多维查询转化为一维检索，但矩形查询在一维编码空间中天然不连续，往往会带来无效点访问的现象。与此同时，学习型索引在一维场景已经表现出较好的效率和空间优势，但在多维空间数据上仍需要与降维方法结合，查询阶段的性能仍存在进一步优化空间。

基于上述背景，本文以“Z 阶曲线降维 + 学习型索引”的思路为基础，围绕空间范围查询中的关键开销无效点访问与碎片化区间带来的多次查询开展研究，主要工作和贡献可以概括为以下三点：

- (1) 设计并实现基于 Z 阶曲线与 PGM 的学习型空间索引结构（ZPI）。

本文设计并实现了 ZPI 索引结构，其核心思路是借助 Z 阶编码将二维空间

点转化为一维 Z 值序列，进而基于 PGM 构建学习型索引以实现数据位置的快速预测。与传统的树型索引相比，该结构摒弃了复杂的层级维护机制，具有更简洁的组织形式和更低的构建开销，能够有效适配大规模点数据场景，为后续范围查询优化奠定了结构基础。

(2) 提出基于最大块分割的范围查询策略，降低无效点访问带来的验证开销。

针对查询区域角点 Z 值区间引入大量无效点的问题，本文设计了最大块分割策略。该方法以最小包含块为起点递归划分查询区域，优先保留完整的标准 Z 阶块并将其组织为有序 Z 值区间集合。在确保查询结果完备的前提下，该策略有效压缩了候选数据范围，显著减少了无效点访问及其带来的验证代价，从而提升了范围查询的整体效率。

(3) 引入边框扩展机制并形成改进算法 $ZPI-RQ^+$ ，缓解区间碎片化。

在最大块分割基础上，本文进一步针对边界区域细分导致的区间数量过多问题，提出了边框扩展机制。通过将查询区域对齐至固定阶 Z 阶边界，减少了曲线在边界处的频繁进出，从而抑制了碎片化区间的产生。同时，算法对候选块进行分类处理，区分直接采纳块与需验证块，进一步降低了验证操作频次。实验结果显示， $ZPI-RQ^+$ 在多种查询场景下性能表现更为稳定，尤其在中大范围查询中优势尤为突出。

综上，本文从索引结构构建与范围查询优化两个层面出发，围绕无效点访问与区间碎片化两个核心问题提出改进方法，并通过实验验证了所提方案在不同数据分布和不同查询率条件下的有效性。

1.4 论文结构

本文围绕基于 Z 阶曲线与学习型索引的空间范围查询优化展开，全文结构安排如下。

第二章主要介绍与本文相关的研究基础与国内外研究进展，包括空间范围查询的基本概念、传统空间索引方法、空间填充曲线（尤其是 Z 阶曲线）相关技术，以及学习型索引在空间数据管理中的典型应用，为后续方法设计提供背景支撑。

第三章首先介绍 ZPI 的整体设计与构建流程，包括 Z 值编码、排序与 PGM 模型建立等核心环节；然后对 ZPI 的构建复杂度进行理论分析；最后通

过实验验证 ZPI 在索引构建时间和空间占用方面的性能优势。

第四章给出本文的基础查询方法。在 ZPI 索引的基础上,针对范围查询中无效点访问较多的问题,提出最大块分割策略,给出对应的 QuadSplit 分割过程及复杂度分析,并在此基础上提出 ZPI-RQ 查询算法,然后通过实验对比分析算法各方面的表现。

第五章在第四章方法的基础上进一步改进查询过程。针对边界区域细粒度分割造成的区间碎片化问题,引入边框扩展机制,并提出完整的范围查询算法 ZPI-RQ⁺;同时对扩展参数与查询性能之间的关系进行分析,给出更稳定的查询流程。提出的方法进行实验评估与结果分析。通过在不同数据分布与不同查询率条件下的对比实验,验证 ZPI-RQ⁺在各方面的表现,并总结实验结果与原因。

最后,对全文工作进行总结,概括本文的主要结论,并对后续可能的改进方向进行展望。

1.5 本章小结

本章作为绪论部分,主要介绍了本文的研究背景与研究意义,说明在空间数据规模不断增长的应用环境下,高效的空间范围查询对位置服务与空间数据管理的重要性。同时,本章对国内外相关研究现状进行了梳理,归纳了传统空间索引、空间填充曲线降维方法以及学习型索引等技术路线在范围查询场景下面临的典型问题,进一步明确了本文关注的核心矛盾——无效点访问与区间碎片化带来的额外开销。

在此基础上,本章给出了本文的出发点与主要工作,概括了论文的三点贡献,并说明了全文的章节组织结构,为后续章节的方法设计与实验验证做好铺垫。

2 相关概念

本章主要介绍本文研究所需的理论基础和相关工作。首先对空间范围查询进行数学定义并列岀全局符号表，接着介绍 Z 阶曲线及相关的编码算法，随后重点阐述学习型索引（尤其是本文依赖的 PGM 索引）的基本原理，最后对现有空间索引与查询方法的优缺点进行对比分析，为后续提出 ZPI 及优化算法提供理论支撑。

2.1 基本定义

2.1.1 空间数据定义

在空间数据库中，最基本的数据形式是点数据。一个二维空间点通常表示为 $C = (x, y)$ ，其中 x 和 y 分别表示点在二维平面上的横坐标和纵坐标。给定一个包含 n 个点的数据集 $D = \{C_1, C_2, \dots, C_n\}$ ，每个点可能还携带额外属性信息，但在空间范围查询中主要关注其空间位置。

在实际应用中，这类数据广泛存在于位置服务系统（LBS）^[43]、地理信息系统（GIS）^[44]、轨迹分析以及兴趣点（POI）^[45]管理等场景。例如地图中的商户位置、共享单车的实时位置、交通传感器记录的坐标等，都可以抽象为二维空间点集合。

需要注意的是，真实空间数据往往呈现明显的分布不均特征。例如人口密集区域的数据点数量远高于偏远地区，这种空间异质性会对索引结构和查询效率产生重要影响。因此，在设计空间索引时，需要充分考虑数据分布对性能的影响。

2.1.2 空间范围查询

给定一个由一系列位置点组成的空间数据集 D ， D 中的每个位置点表示为 $C = (C.x, C.y)$ ，其中 $C.x$ 和 $C.y$ 分别表示 C 的横纵坐标。一个查询范围 R 为一个由左上坐标 $R.C_0 = (x_0, y_0)$ 和右下坐标 $R.C_3 = (x_3, y_3)$ 表示的空间区域。

空间范围查询指给定一个空间数据集 D 和一个查询范围 R ，查询返回 D 中所有满足 $C \in R$ 的位置点。形式化定义如下：

$$Q(R) = \{C \in D \mid x_0 \leq C.x \leq x_3 \wedge y_0 \leq C.y \leq y_3\} \quad (2.1)$$

2.1.3 符号定义

为便于后文算法描述，表 2.1 列出了本文通用的数学符号及其含义。

表 2.1 符号解释表

符号	描述
D	一个数据集
C	一个数据点
R	查询区域，由 $C_1=(x_1, y_1)$ 、 $C_3=(x_3, y_3)$ 组成
m	查询区域需要扩展的阶数
R^m	m 阶扩展后查询区域
$R.b_l, R.b_t, R.b_r, R.b_b$	分别为查询区域的左边框线、上边框线、右边框线、下边框线
k	Z 阶曲线的阶数
B	一个 Z 阶曲线区域
$B_{00}, B_{01}, B_{10}, B_{11}$	分别为一个块四分后的左上块、右上块、左下块、右下块
$Z_B^0, Z_B^1, Z_B^2, Z_B^3$	一个块的左上角点，右上角点，左下角点和右下角点的 Z 值
C_0, C_1, C_2, C_3	一个区域的左上角点，右上角点，左下角点和右下角点
MCB	包含查询区域的最小 Z 阶区域
L_v	待验证列表，存储待验证的段
L_f	免验证列表，存储无需验证的段
r	查询率，查询框占整个数据空间的大小

2.2 Z 阶曲线

空间填充曲线可将多维数据高效地映射到一维空间，同时保持数据点的空间局部性，即空间相近的数据在映射后的一维数据也尽可能地相近，它在数据库索引、图像处理、点云序列化和查询优化中发挥关键作用，常见的空间填充曲线有 Z 阶曲线、Hilbert 曲线等。

Z 阶曲线（ Z -order curve），也称为莫顿曲线（Morton curve），是一种基于递归空间划分的空间填充曲线。 Z 阶曲线的阶 n 代表着对空间划分的粒度， n

阶曲线将整个空间划分为 $2^n * 2^n$ 的网格^[48]。Z 阶曲线经过每个网格一次且仅一次，每个网格在曲线的序号为其编码值，本文简称为 Z 值。

Z 阶曲线的编码较为直观，最常见的方法为位交错法^{[46][47]}。其通过将 $C.y$ 和 $C.x$ 逐位交错即可获得位置点 C 的编码。例如图 2.1 (a) 三阶曲线中查询框左上角点的坐标为 $C_0=(1,2)$ ，其对应的编码 $C_0.Z=(001001)_2=9$ ，即 $C_0.Z=9$ ，对应曲线中的第 9 个位置。

Z 阶曲线的查询同样较为直观，其左上角点和右下角点的编码对应的范围即为查询对应的范围。给定图 2.1 (a) 左上角点 $C_0=(1,2)$ 和右下角点 $C_3=(6,5)$ ，通过编码知 $C_0.Z=9$ 和 $C_3.Z=54$ 。即查询框内的所有点的 Z 值必在 9 和 54 范围内。尽管如此，Z 阶曲线存在的较为直观的不足：该范围内可能包含大量查询框外的无效点，从而不可避免地影响查询性能。

除此之外，近年来有其它 Z 阶曲线编解码算法的研究，可有效提升编解码的效率。此外，尽管 Hilbert 曲线具有更高的内聚性，但考虑到其编解码算法和查询算法相对较为复杂，因此目前的多数算法多采用 Z 阶曲线实现。

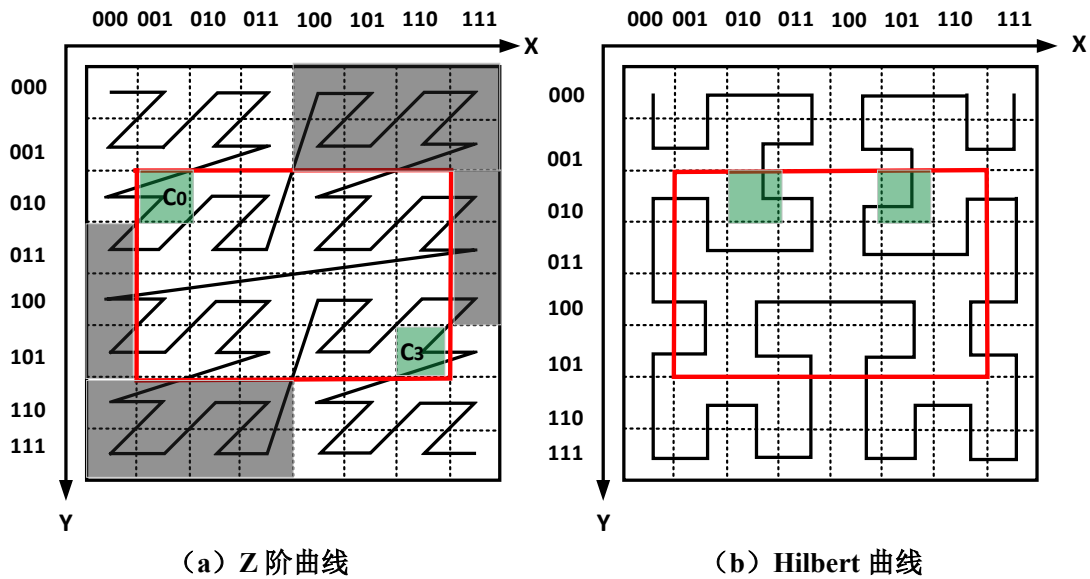


图 2.1 3 阶 Z 阶曲线和 Hilbert 曲线

2.3 传统空间索引方法

2.3.1 R-tree 及其变种

1984 年, Guttman 提出了一种专门面向多维空间数据的索引结构——R-tree (R 树)。这一结构自诞生以来, 便成为空间数据库与地理信息系统 (GIS) 领域最具影响力的索引方法之一, 被广泛应用于各类空间数据管理系统中。从

设计理念来看, R-tree 可视为 B 树在高维空间的拓展与泛化, 其核心思想是利用最小外接矩形 (Minimum Bounding Rectangle, MBR) 对空间对象进行逐层聚合, 构建一棵高度平衡的多路查找树。

当系统接收到一个空间范围查询请求时, R-tree 采用自顶向下的递归遍历策略执行查找。算法从根节点开始, 逐一判断当前节点存储的 MBR 是否与给定的查询区域 R 存在交集。如果存在交集, 则继续向下访问相应的子节点; 如果不存在交集, 则直接剪枝, 放弃对该分支的进一步探索。这一剪枝机制通过提前排除大量无关的空间区域, 有效减少了需要访问的节点数量, 从而加速了查询过程。

在 R-tree 的基础上, 研究人员提出很多种变体来改进它的表现, 其中最具有代表性的就是 R*-tree。R*-tree 对原始 R-tree 的节点分裂策略和插入规则做了系统性的调整, 通过综合考量节点覆盖面积、重叠区域以及节点周长等多项因素, 旨在最小化节点 MBR 之间的重叠程度。这一优化显著降低了查询路径上需要遍历的分支数量, 尤其在中高密度数据场景下表现更为出色。

2.3.2 四叉树与 k-d 树

四叉树作为一种经典的空间索引结构, 其核心思想建立在规则空间划分基础之上。在二维场景里, 四叉树的根节点对应整个数据空间, 当某个节点区域里包含的数据点数量超出预设容量阈值时, 该节点会被严格等分为四个面积相同的子象限, 分别对应左上、右上、左下、右下四个方向, 这一递归分割过程持续进行, 直至所有叶子节点内的数据量均满足容量约束为止。四叉树的优势在于结构直观、实现简单, 且与地图缩放等应用场景天然契合。然而, 其固定的等分策略也带来了固有缺陷: 当数据分布呈现明显倾斜时 (如城市中心密集而郊区稀疏), 四叉树容易产生深度差异巨大的子树, 部分分支路径过长, 导致树结构失衡, 查询效率的稳定性受到影响。

针对四叉树因固定等分导致的失衡问题, k-d 树引入了一种更为灵活的自适应划分策略^[50]。与四叉树不同, k-d 树是一棵严格的二叉树, 其划分方式具有两个显著特征: 一是划分维度的轮换交替, 通常按照 X 轴、Y 轴、再 X 轴的顺序逐层切换; 二是划分位置的数据驱动, 每次选择当前维度上所有数据点的中位数作为分割超平面。这种基于中位数的切分方式从数学上保证了构建出的二叉树具有良好的平衡性, 有效避免了四叉树中可能出现的深度失衡现象, 正因如此, k-d 树在静态数据集上的点查询和最近邻查询中表现优异^{[51][52]}

2.3.3 网格类索引方法

与 R 树和四叉树这类依赖多层指针跳转的树状结构不一样，网格类索引走的是更为扁平的空间映射设计路线。其核心思想是通过预设的边界将连续的多维数据空间切割为离散的网格单元，并借助哈希函数或数学映射，将数据点直接定位到对应的网格桶中。这种设计避免了树结构的逐层遍历，理论上可实现常数时间的定位开销。

均匀网格索引是这一思路最直观的表现，它把数据空间等分成大小一样的矩形网格，每个数据点借助简单的坐标运算就能映射到对应网格，做范围查询的时候，只需要定位查询框所覆盖的网格，再从里面筛选结果，这种机制实现起来特别简单，构建速度也快，在小规模或者均匀分布的数据集上表现不错，可是真实世界的空间数据常常呈现出高度倾斜的分布特点。在这种场景下，均匀网格索引的刚性划分暴露出三个核心问题：空间利用率不高，没有数据的区域会产生大量空网格，带来内存浪费，其次热点区域拥堵，密集区域的数据堆积在少数网格里，导致网格内退化成线性扫描，查询效率急剧下降。

针对均匀网格索引因刚性划分引起的数据倾斜问题，等深网格索引引入了一种自适应的调整思路，和均匀网格索引追求网格面积相等不同，等深网格索引的核心目标是让网格内数据量大致相等。在构建阶段，等深网格索引基于数据在各维度上的累积分布函数或者利用分位数计算，动态地确定网格的划分边界，这一机制使得数据密集区域被划分成多个面积较小但数量均衡的网格，而数据稀疏区域则保留面积较大的网格，从而在一定程度上缓解了桶内数据量不均衡的情况。

2.4 学习型索引

2.4.1 学习型索引基本思想

传统 B+ 树的查找过程可以看作是一个映射函数： $\text{Key} \rightarrow \text{Position}$ 。对于排序数组，这个映射函数就是累积分布函数的近似。如果我们可以用一个机器学习模型拟合这 CDF，即 $\text{Pos} = \text{model}(\text{Key})$ ，那么我们就可以直接通过计算预测数据的位置，从而替代 B+ 树的层级遍历。学习型索引的核心思想是将索引结构视为数据分布的函数模型^[55]。通过学习数据键值与其存储位置之间的映射关系，可以利用函数预测代替树结构导航^[56]。

2.4.2 PGM 索引

在学习型索引的发展早期, Kraska 等人提出的 RMI (递归模型索引) 开创性地将机器学习模型引入索引结构设计, 为这一领域奠定了基础。在 RMI (递归模型索引) 开创了学习型索引这一新范式之后, 研究者们开始关注其在实际应用中暴露的局限性。RMI 采用多层神经网络模型^[59], 虽然在某些场景下表现优异, 但存在两个主要问题: 一是神经网络的训练需要大量计算资源, 构建开销较高; 二是模型缺乏理论上的最坏情况误差保证, 难以满足数据库系统对性能可预测性的需要。这些局限性催生了新一代学习型索引的研究, 其中最具有代表性的成果之一便是 Ferragina 和 Vinciguerra 提出的 PGM 索引 (Piecewise Geometric Model index)^[36]。

PGM 的核心包括 Piece-Rise Linear Approximation model (PLA-model) 和 PGM-index 两部分^[58]。PLA 模型是 PGM 的基础组件。它通过贪心算法对键-位置映射关系进行分段线性拟合, 在保证每段拟合误差不超过预设阈值 ϵ 的前提下, 生成最优数量的线性片段^[54]。这一机制实现了模型复杂度与预测精度的平衡。PGM-index 在 PLA 模型基础上构建层次化结构: 将第一层生成的每个分段的起始键收集起来, 再次应用 PLA 模型, 递归进行直至顶层仅剩一个分段。这种多层设计使得查询可以通过逐层模型预测快速定位, 类似于 B+树的多级导航, 但用模型计算替代了指针遍历。

(1) 最佳分段线性近似模型 (Optimal PLA Model)

给定一个包含 N 个有序键值的集合 $K = \{k_1, k_2, \dots, k_N\}$, 其中 k_i 在数组中的物理位置为 i 。PGM 索引的目标是寻找一组最少数量的线性线段 $S = \{s_1, s_2, \dots, s_m\}$ 来覆盖整个数据集。对于每一个线段, 其由三个参数定义: 起始键 key_{start} 、斜率 a_j 和截距 b_j 。该线段覆盖的键值 k 满足如下预测模型: $Pos_{pred}(k) = \lfloor a_j \cdot k + b_j \rfloor$, 其中 $Pos_{pred}(k)$ 是键 k 的预测位置^[53]。

PGM 索引最显著的特征是引入了用户预定义的误差界限 ϵ 。算法保证对于线段 S_j 覆盖范围内的任意键 k_i , 其真实位置 $Pos_{real}(k_i) = i$ 与预测位置 $Pos_{pred}(k_i)$ 之间的绝对误差不超过 ϵ : $|Pos_{real}(k_i) - Pos_{pred}(k_i)| \leq \epsilon$, 这意味着, 所有被线段 S_j 盖的数据点, 在二维平面上都必须严格落在由直线 $y = a_j x + b_j$ 上下偏移所形成的误差管道 (Error Tube) 内 (如图 2.2 所示)。

(2) 自底向上的构建与查询机制

PGM 采用一种基于流式处理的贪心算法进行构建。算法按顺序扫描数据

集，维护当前线段的凸包性质。当新加入的数据点无法被当前的 ϵ -误差管道包含时，算法判定当前线段终止（如图 2.2 中的 Breakpoint），并立即以该点为起点开启一个新的线性分段。由于该算法的时间复杂度为 $O(N)$ ，因此 PGM 具有极高的构建效率，非常适合处理海量空间数据的 Z 值序列。

为了快速定位键值属于哪一个线段，PGM 采用了类似于 B+树的递归结构。底层（Level 1）存储实际拟合数据的线段集合 S；上一层（Level 0）则提取 Level 1 中每个线段的起始键作为新的输入数据，再次构建 PGM 模型。这一过程递归进行，直到顶层模型的大小足以放入 CPU 缓存。

在执行查找操作时，PGM 自顶向下遍历。当查询到叶子层具体的线段 S_j ，首先通过公式计算预测位置 $y = a_j x + b_j$ 。根据 ϵ 的保证，目标键值的真实位置必然在区间 $[p - \epsilon, p + \epsilon]$ 内。因此，只需在这个长度为 $2\epsilon + 1$ 的极小范围内进行二分查找即可找到目标数据。

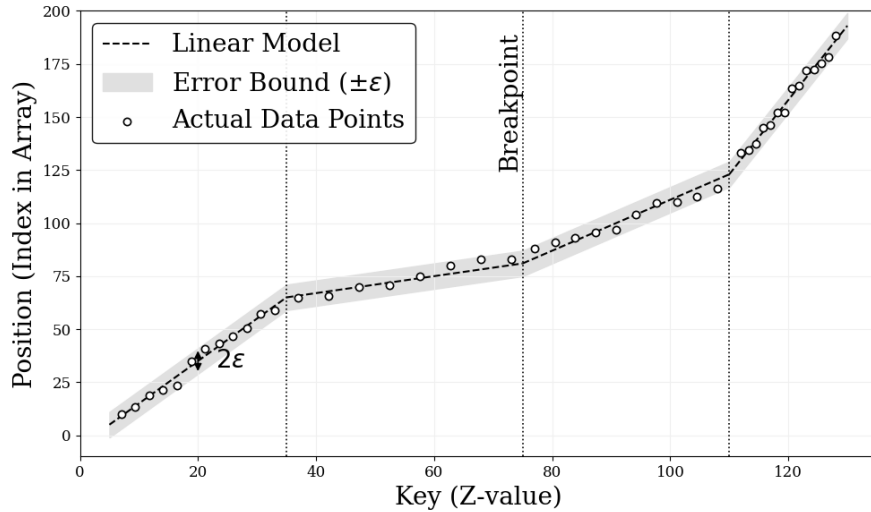


图 2.2 分段线性模型图

（3）PGM 在 ZPI 中的优势

选择 PGM 作为 ZPI 索引的底层一维索引，主要基于以下考量：

确定性的查询性能：与神经网络模型不同，PGM 提供了严格的误差界限 ϵ 。在范围查询中，这意味着我们无需进行大范围的推测性扫描，能够精确控制 I/O 开销。

对 Z 值分布的适应性：多维数据经 Z 阶曲线降维后，Z 值序列通常呈现出局部聚集、全局稀疏的特征。PGM 能够自适应地调整线段斜率（斜率即代表局部数据密度），在数据密集处使用短线段，在稀疏处使用长线段，从而实现了对非均匀分布的高效拟合。

极低的空间占用：PGM 仅需存储线段参数（两个浮点数和一个整数），相比于 R-Tree 需要存储大量的 MBR 坐标和指针，PGM 的空间压缩率通常可达几个数量级，这为在内存中处理大规模空间索引提供了可能。

2.5 本章小结

本章系统构建了本文研究的理论框架。首先，通过严格的数学定义明确了空间范围查询的目标与挑战，特别是说明了无效点对查询性能的负面影响。其次，深入剖析了 Z 阶曲线的构造机理，指出了其在保持空间局部性与区间连续性之间的内在矛盾，为第四章最大块分割算法的设计提供了理论动因。最后，通过对传统索引与前沿学习型索引的对比分析，论证了基于 PGM 构建 Z 阶索引（ZPI）的可行性，并指出了现有线性化策略在范围查询优化方面的不足。这些理论分析不仅揭示了问题的本质，也指明了本文后续章节通过最大块分割与边框扩展策略来优化查询效率的技术路线。

3 ZPI 索引的设计与实现

3.1 引言

在空间数据库与地理信息系统领域，空间范围查询是最基础且应用最广泛的操作之一。随着移动互联网与物联网技术的快速发展，空间数据的规模呈爆发式增长。面对海量数据，传统树型空间索引（如 R 树、R*树及四叉树）因构建开销大、层级结构深、节点频繁分裂等问题，在高性能实时查询场景下日益显得力不从心，亟需探索新的索引技术路径。

近年来，学习型索引的提出为索引设计带来了全新思路。该方法通过机器学习模型拟合数据分布，利用函数模型直接预估数据存储位置，以替代传统的层级遍历，从而有效压缩索引层级、减少 I/O 访问次数。在一维数据场景下，学习型索引已展现出优于传统 B 树的性能潜力。然而，二维空间数据缺乏天然的线性顺序，无法直接应用一维学习型索引。

Z 阶曲线（Z-order curve）因其编码简便、局部保持性良好等优点，成为连接多维空间与一维学习型索引的重要桥梁。通过 Z 阶编码将二维坐标映射为一维 Z 值序列，可以在该序列上构建学习型索引，从而间接支持多维空间查询。PGM 索引（Piecewise Geometric Model index）作为一种高效的一维学习型索引，具备确定性误差界限、构建速度快、空间占用低等优点，非常适合作为 Z 值序列上的底层索引结构。

基于上述背景，本章设计并实现了一种面向空间点数据的 ZPI（Z-order curve + PGM Index）混合索引结构。ZPI 的核心思路是：首先利用 Z 阶曲线将二维空间点映射为一维 Z 值序列，并按照 Z 值排序，然后在该有序序列上构建 PGM 索引，实现数据位置的快速预测。同时，ZPI 维护 Z 值与原始坐标之间的映射关系，以支持空间查询时的坐标获取与验证。

3.2 ZPI 索引结构

多维空间数据因维度耦合难以直接进行线性排序，因此难以直接基于 PGM 在多维数据上构建学习型索引。为此，我们通过 Z 阶曲线编码算法将每个分区内的多维数据映射为一维 Z 值，将空间数据转化为可排序的线性序列。

3.2.1 索引组成与构建流程

为了能利用多维数据降维与学习型索引查询的优势，本章提出了一种名为 ZPI 的混合索引结构。这个结构的核心在于将 Z 阶曲线与 PGM 学习型索引进行有机结合。构建 ZPI 索引工作流程分为三个阶段：首先，借助 Z 阶曲线将二维空间数据降维，转化为一维顺序的 Z 值序列并按照 Z 值排序，这样就化解了多维数据在排序上的天然缺陷。然后建立 Z 值与原始二维数据的映射表，这一步是为了提高最后查询的效率。最后引入 PGM 索引对生成的 Z 值分布进行构建索引，利用其学习能力实现数据存储位置的快速预估。ZPI 的整体框架与各模块间的协作关系如图 3.1 所示。

(1) Z 值数组（记为 A_z ）

如图 3.1 左侧流程所示，原始空间数据集 $D = \{C_1, C_2, \dots, C_n\}$ 首先经过 Z 阶曲线编码进行降维处理。对于每个二维坐标点 $C_i = (x_i, y_i)$ ，通过编码算法计算其 Z 值 $Z_i = Z_Encode(x_i, y_i)$ 。所有 Z 值按升序排列后形成一维有序数组 $A_z = [Z_1, Z_2, \dots, Z_n]$ ，应图中底层表格的 ZCode。

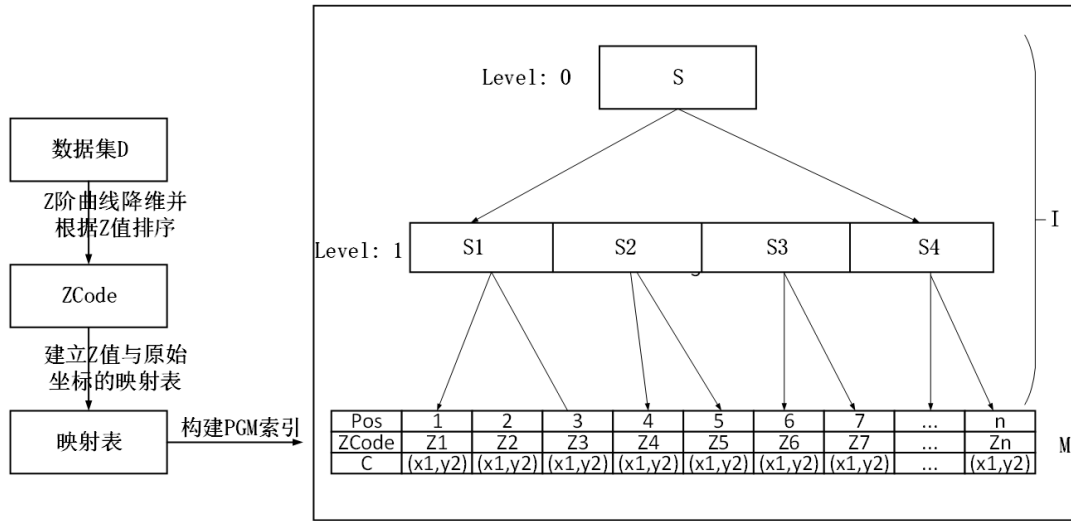


图 3.1 索引构建结构图

为了使最后范围查询更高效，我们将数组严格按 Z 值排序，使得二分查找时能够快速锁定目标区间。在物理层面，数组以连续内存形式存储，这不仅能充分利用 CPU 缓存预取能力来加速顺序扫描，还通过数组下标（Pos）建立了与坐标映射表的关联，这样能使计算机更快速找到原始空间数据。

(2) 坐标映射表（记为 M ）

坐标映射表是连接一维 Z 值与二维原始空间数据的桥梁，如图 3.1 底层表格所示，该表维护 Z 值到原始坐标 (x, y) 的映射关系。对于 A_z 中第 i 个元素 Z_i ，映射表记录其对应的原始坐标 $C_i = (x_i, y_i)$ 。这是一种高效的查表机制来替代传统的解码运算，在空间范围查询的验证环节，当 PGM 索引锁定候选 Z 值区间后，索引不用通过对 Z 值进行解码来获取坐标，而是直接通过数组下标 (Pos) 查访问预存的坐标映射表，快速得到对应的二维坐标。随后仅需执行一次简单的坐标比对，即可判断数据点是否位于查询区域内。这一设计利用存储空间换取计算时间，省去了 Z 阶曲线解码的复杂过程，从而有效压缩了查询响应时间。

(3) PGM 层次索引 (记为 I)

PGM 索引是在 A_z 上构建的自适应分层导航结构，如图 3.1 右侧层次结构所示。该结构采用自底向上的递归构建方式，形成多级模型树：

Level 0 (顶层根节点)：对应图中标记为“s”的根节点，存储全局统计信息或顶层模型参数，用于快速粗略定位。

Level 1 (分段模型层)：对应图中 S1-S4 等中间节点，将 Z 值空间划分为若干连续分段。每个分段 S_j 由一个分段线性近似模型描述，记录该段的起始 Z 值 Z_{start}^j 、线性斜率 a_j 和截距 b_j ，满足位置预测公式 $\text{Pos} = a_j \bullet Z + b_j \pm \varepsilon$ ，其中 ε 为预设误差界。若 Level 1 节点数量过多，则提取各段起始 Z 值作为新的数据点集，递归构建上层 PGM 模型，直至顶层仅剩单一节点。

这种层次化设计使得查询操作可通过自顶向下的导航快速定位到目标分段：首先在顶层通过线性计算确定候选分段，然后逐层下推，最终在底层通过范围内的局部搜索精确定位数据。相较于传统 B+树的 $O(n \log n)$ 复杂度，PGM 的查找复杂度可达 $O(\log \log n)$ ，且空间占用更小。

在与 ZM 索引的对比中，ZPI 所采用的 PGM 架构展现出多维度的性能优势。首先，在查询可靠性方面，PGM 具备确定性误差边界，能够保证最坏情况下的查询性能稳定可控；相比之下，基于神经网络的 RMI 由于缺乏理论上的最坏情况保证，其查询延迟存在不确定性。其次，在资源消耗层面，PGM 的结构极为精简，每层仅需存储斜率和截距两个参数，而 RMI 需要维护多层网络的权重矩阵，导致内存开销提升了一个数量级。再者，从构建效率来看，PGM 基于贪心算法线性扫描即可完成构建，无需像 RMI 那样进行耗时的反向传播迭代训练，因此构建速度更快。综合这些优势，ZPI 在需要频繁重建索引的动态空间数据场景中具有更强的适配性。

3.2.2 Z 阶曲线编码

目前有众多可用的 Z 阶曲线的编解码算法。位交错是目前最简单也是最常用的 Z 阶曲线编解码算法，其可通过逐位交叉输入坐标将多维数据降为一维。但其复杂度为 $O(k)$ ， k 为需要编码的阶数。最近的工作，VLUTg-ZE^[49]，采用高效的 g 阶查找表，可通过一次查表直接获得 g 阶编码，可将复杂度降低到 $O(k/g)$ ，具有更高的编码效率。考虑到这一点，本文采用 VLUTg-ZE 作为本文默认的编码算法。

3.3 索引构建

算法 1 给出了 ZPI 索引的完整构建过程。

算法 1: Build_ZPI(D)

输入: D: 数据集

输出: ZPI: ZPI 索引

1. $Az \leftarrow \emptyset$
 2. for each coordinate $C \in D$ do
 3. $Zp \leftarrow Z_Encode(C.x, C.y)$
 4. $Az \leftarrow Az \cup \{Zp\}$
 5. sort Az in ascending order
 6. $ZPI \leftarrow \text{build PGM with } Az$
 7. return ZPI
-

算法 1 中对每个数据点进行 Z 阶编码（第 2-4 行），本文采用 VLUTg-ZE 算法，通过预计算的 g 阶查找表将编码复杂度从 $O(k)$ 降低至 $O(k/g)$ ，其中 k 为编码阶数。然后对 Z 值数组进行排序（第 5 行），排序是构建过程中时间开销最大的阶段。最后基于排序后的 Z 值数组构建 PGM 索引（第 6 行），PGM 采用贪心算法线性扫描数据集，维护当前线段的凸包性质，当新数据点无法被当前 ϵ -误差管道包含时，终止当前线段并开启新分段。

3.4 复杂度分析

ZPI 索引构建过程包含三个主要阶段，关于时间复杂度的分析如下：Z 阶编码阶段，对于 n 个数据点，单个点的编码复杂度是 $O(k)$ ，这里 k 代表编码阶数。如果采用 VLUTg-ZE 算法来做优化，借助粒度为 g 的查找表，可以把单点复杂度降低到 $O(k/g)$ ，所以整个编码阶段的复杂度就是 $O(n \cdot k/g)$ 。排序阶段，对生成的 n 个 Z 值做排序，假如使用快速排序这类比较排序算法，平均时间复杂度为 $O(n \log n)$ ，PGM 模型构建阶段，首先通过一次线性扫描遍历所有 ZCode

来构建底层 PLA 模型，复杂度为 $O(n)$ 。假设分段数为 s ，并且 s 远小于 n ，那么上层索引的构建复杂度大概在 $O(s \log s)$ ，可以近似看作 $O(n)$ ，整体来看，这个阶段的总复杂度能控制在 $O(n)$ 量级，综合以上，ZPI 索引的总体构建复杂度为 $O(n \cdot k/g + n \log n)$ 。由于编码阶数 k 和查找表粒度 g 一般取为常数，这个复杂度主要受数据规模 n 和排序开销 $n \log n$ 影响。

空间复杂度方面，降维需要存储 Z 值，空间复杂度为 $O(n)$ ；为加速验证，保留原始坐标，空间复杂度为 $O(n)$ 。PGM 模型存储：设 PGM-index 包含 s 个分段，每个分段存储斜率和截距两个浮点数，空间复杂度为 $O(s)$ 。总的空间复杂度为 $O(n+s)$ 。

3.5 实验分析

本文所有的实验均在一台计算机上进行，该计算机 CPU 为 Intel Core Ultra 5-125H，频率为 4.5GHz，14 核 18 线程，内存为 16GB，能够提供稳定的计算环境。实验环境基于 Ubuntu 24.04 操作系统，采用 C++ 编程语言实现，代码使用 GCC 13.3.0 编译器编译。

3.5.1 数据集及查询

本文使用以下四个数据集：

GeoName 数据集^[60]：该数据集是从 GeoName 全球地理数据库提取的，包含 11061987 条地理位置记录。GeoName 整合了多种官方来源的地理数据，覆盖全球范围内，包含大陆、国家、行政区划、城市及地标等的多种信息。在空间分布上，该数据集呈现出显著的空间复杂性：人口稠密地区（如大城市群、沿海地带）数据点高度密集，而自然条件恶劣的区域（如沙漠、海洋、极地）则数据稀疏。这种非均匀分布特性能够评估索引在各种复杂数据分别下的性能。

OpenStreetMap (OSM) 数据集^[61]：我们从 OpenStreetMap 提取中国区域的地理数据，共 5284596 个位置点。OSM 是一个全球协作的地理数据平台，其中中国区域数据反映中国地理分布特征是东部人口稠密地区数据密集，西部和北部人口稀疏区域数据相对稀疏，而且包含丰富的兴趣点（POI）类型，如交通设施、商业场所、公共服务设施等。

均匀分布数据集 (Uniform)：包含 2000 万个点，在单位正方形区域 $[0,1] \times [0,1]$ 上服从均匀分布。该数据集空间分布完全均匀，没有任何聚集或稀疏区域，作为理想状态，用于评估方法在最理想分布下的性能。

正态分布数据集 (Norm): 包含 2000 万个点, 服从二维标准正态分布, 均值为 0, 方差为 1。该数据集空间分布呈现中心聚集、外围稀疏的特征, 模拟现实世界中常见的聚集分布模式 (如城市中心密集、郊区稀疏)。

表 3.1 数据集信息

Property	Geoname	OSM	Uniform	Norm
元素个数	11061987	5284596	20000000	20000000
数据集大小	198MB	131.9MB	1GB	1GB

为每个数据集, 我们计算数据的空间范围 $R_x = [x_{\min}, x_{\max}]$ 和 $R_y = [y_{\min}, y_{\max}]$, 设定网格分辨率 $r = 8192$, 将每个维度均匀划分为 r 个区间, 计算网格宽度:

$$w_x = \frac{x_{\max} - x_{\min}}{r}, \quad w_y = \frac{y_{\max} - y_{\min}}{r}, \quad \text{对于任意数据点 } (x_i, y_i), \text{ 通过线性映射计算}$$

$$\text{其对应的整数网格索引 } (I_x^i, I_y^i), \quad I_x^i = \left\lfloor \frac{x_i - x_{\min}}{w_x} \right\rfloor, \quad I_y^i = \left\lfloor \frac{y_i - y_{\min}}{w_y} \right\rfloor.$$

3.5.2 实验设置

为验证本文索引结构和算法的性能, 本文选择传统空间索引结构 R*-tree^[11] 和众多代表性的学习型索引 MLI^[41]、ZM^[39]、LISA^[42]、UG^[23]、EDG^[63]、Flood^[57]、SPRIG^[62] 等作为对比算法。

R*-tree^[11]: 是一种用于高效处理多维空间数据索引的数据结构, 是对 R-tree 改进, 通过改进节点分裂策略和插入算法, 提高了查询效率。使用 Boost Geometry Index 库 R*-tree 实现, 最大叶节点设置为 128;

MLI^[41]: 选择一组参考点, 利用数据与参考点之间的距离计算一维值, 从而实现降维, 然后构建索引, 实现查询;

ZM^[39]: 通过 Z 阶曲线将多维空间数据映射到一维空间, 再利用神经网络学习 Z 值分布规律, 构建轻量级模型预测数据位置, 实现高效的空間查詢;

LISA^[42]: 利用网格划分数据, 并将其映射为可搜索的数据布局, 构建分段线性模型实现查询;

UG^[23]: Uniform Grid Index (均匀网格索引) 是一种基于固定划分的空间索引结构。其核心机制是将目标空间分割为若干尺寸相同的网格单元, 每个单元负责存储落入该区域内的数据点 (如经纬度坐标)。该方法实现简单、查询高

效，在地理位置数据检索中应用广泛。

EDG^[63]：Equal-Depth Grid Index（等深网格索引）则采用了一种自适应的划分策略。与 UG 关注网格面积均衡不同，EDG 以网格内数据量均衡为目标，通过调整网格边界使每个单元包含大致相当的数据点。这种设计有效缓解了均匀网格在数据分布不均场景下的性能瓶颈，特别适用于 Foursquare 等 LBSN 数据中的热点区域查询（如城市中心密集 POI）。

Flood^[57]：一种多维内存读取优化的学习型索引，通过自动调节索引结构和数据存储以适应数据布局。

SPRIG^[62]：使用空间插值函数直接拟合多维数据分布，可以直接对原始的多维空间数据进行建模和预测。

3.5.3 索引构建时间分析

为了评估索引构建效率，本节对比了 ZPI 索引、R*-tree、ZM 索引、MLI 索引、LISA 索引、UG 索引、EDG 索引、Flood 索引、SPRIG 索引在不同数据集上的构建时间。各数据集下，算法对比结果如图 3.2 所示。

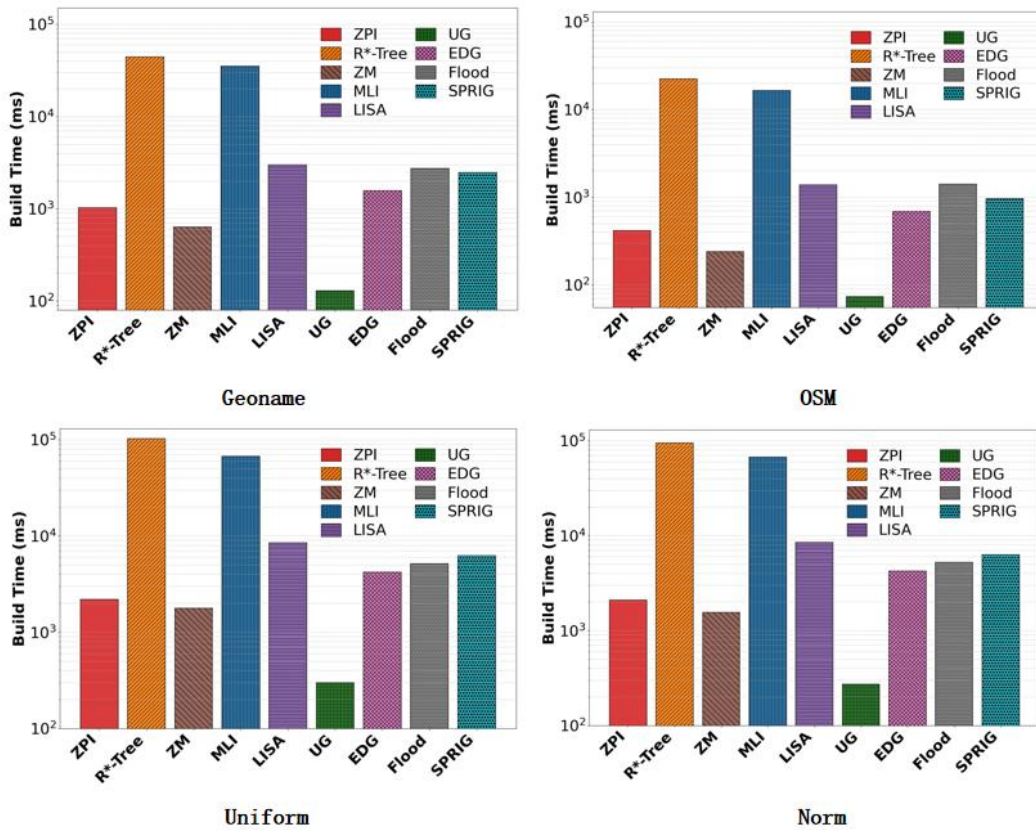


图 3.2 索引构建时间对比图

ZPI 索引的构建过程主要包含 Z 值编码、数据排序、PGM 模型构建几个阶段, 其中数据排序是时间开销的主要的, 通常占总构建时间的 40%-55%。通过图 3.2 可知, 相较于传统索引, ZPI 比 R*-Tree 的构建速度快 40-50 倍, 一个重要的原因是 R*-Tree 需要进行多次节点分裂、合并和重平衡操作, 构建过程复杂且耗时, 而 ZPI 通过 Z 阶曲线降维后只需要一次线性扫描就可以构建 PGM 索引。ZPI 索引的构建时间高于 UG 索引, 原因是 UG 采用简单的空间网格划分策略, 仅需根据数据空间范围计算网格参数, 然后通过函数将数据点直接映射到对应网格单元, 不需要对所有数据全局排序和复杂模型训练。

与其他学习型索引相比, ZPI 索引构建快于 LISA、MLI、Flood 和 SPRIG 等学习型索引结构, LISA 构建时间较 ZPI 高出 20%至 30%, 原因是 LISA 采用网格划分与勒贝格测度双重映射, 要先构建网格统计每个单元数据分布, 然后再计算降维等。MLI 基于 iDistance 方法, 需要计算各个数据点与参考点之间的距离然后选择参考点, 整个过程计算量大。Flood 作为多维内存优化索引, 涉及数据重排、列压缩和结构自适应调整等复杂预处理, 构建流程最为繁重。SPRIG 使用空间插值函数拟合多维数据分布, 高维插值计算与参数优化的复杂度高于 PGM 的线性分段。

3.5.4 索引空间分析

为了评估索引的空间开销, 本节对比了 ZPI 索引、R*-tree、ZM 索引、MLI 索引、LISA 索引、UG 索引、EDG 索引、Flood 索引、SPRIG 索引算法在四个数据集上的索引空间占用情况 (单位: KB)。

表 3.2 索引空间占用对比表

数据集	ZPI	R*-Tree	ZM	MLI	LISA
Geoname	173062	264304	173057	172885	173021
OSM	82752	126759	82737	82614	83264
Uniform	312522	459036	312526	312541	313049
Norm	312553	451685	312553	312543	313046

续表 3.2 索引空间占用对比表

数据集	UG	EDG	Flood	SPRIG
Geoname	172928	172929	173234	181976
OSM	84037	84041	83183	91704
Uniform	312734	312736	312851	321632
Norm	312734	312736	312878	321632

如表 3.2 可知, 在空间开销方面, ZPI 索引展现出显著优势。与 R*-Tree 相

比，ZPI 的索引空间开销在所有数据集上减少了 30%至 40%，这主要是因为两者架构的本质不同，R*-Tree 需维护大量内部节点、边界框及指针信息，结构比较复杂，而 ZPI 作为一维学习型索引，仅需存储排序后的 Z 值数组、原始坐标以及 PGM 模型的少量分段参数（斜率和截距），数据量比较少。

与 ZM、MLI、LISA、UG、EDG、Flood 等索引相比，ZPI 的空间占用基本相当，原因是这些算法的思想相似，都是通过降维将多维索引问题转化为一维索引问题，从而消除传统空间索引的难以排序的问题。它们的存储开销均集中于原始数据、降维键值和模型三部分，且都避免了 R*-Tree 的节点和指针冗余。因此，尽管具体实现各有侧重，但它们在空间效率上呈现出一定的相似性。

3.6 本章小节

本章针对多维空间数据难以直接应用一维学习型索引的问题，设计并实现了 ZPI 混合索引结构。ZPI 的核心思想是：借助 Z 阶曲线将二维空间点映射为一维 Z 值序列，从而将多维索引问题转化为一维索引问题；在此基础上，引入 PGM 学习型索引对 Z 值序列构建层次化模型，实现数据存储位置的快速预测。

理论分析与实验结果表明：ZPI 索引在索引构建时间和内存占用方面均表现优异，构建速度比 R*-tree 快 40-50 倍，空间占用减少 30%-40%。

4 基于 Z 阶曲线最大块分割的空间范围查询算法

4.1 引言

在空间数据库与地理信息系统（GIS）等领域中，空间范围查询是一项至关重要的基础技术。其核心任务是在给定一个特定区域 R 后，迅速筛选出位于该范围内的全部空间数据。然而，随着移动终端和遥感技术的普及，空间数据的规模呈现出爆发式增长趋势。在这种海量数据背景下，诸如 R 树、 R^* 树以及四叉树等经典的树型索引结构，由于构建开销大、树层级深以及节点频繁分裂等问题，在满足高性能实时查询方面显得力不从心。

近年来，学习型索引的出现为索引设计带来了全新的视角。该方法通过机器学习模型捕捉数据的分布特征，利用函数模型直接估算数据位置，从而有效压缩索引层级、减少 I/O 访问次数。第三章构建的 ZPI 索引正是这一思路的具体实践：通过 Z 阶曲线将二维空间点映射为一维 Z 值序列，并利用 PGM 学习型索引实现数据位置的快速预测，在构建效率和空间占用方面均表现优异。

然而，Z 阶曲线在实际应用中并非尽善尽美。尽管它能将二维坐标转换为一维 Z 值，并在一定程度上保留空间邻近关系，但在执行范围查询时，一个显著的缺陷暴露了出来：查询窗口映射到 Z 值空间后，往往会变成一个跨度很大的连续区间，其中混杂了大量实际并不在查询框内的无效数据点。即使借助 ZPI 索引高效的一维检索能力能够快速锁定该区间，后续仍需对大量无效点进行逐个验证，这无疑增加了额外的计算负担。当查询区域的形状不规则或边界复杂时，这个问题会更加严重。

为了解决这一瓶颈，研究人员通过将查询窗口进行拆分处理，把一个大的 Z 值区间切分成若干个更紧凑的子区间，从而降低无效点的干扰。但如果拆分过于粗放（如简单网格划分）或递归过深，又会产生大量细碎的小区间，导致索引查询次数激增，反而拉低整体性能。因此，如何在剔除无效数据和控制查询次数之间找到平衡，是提升查询效率的关键所在。

针对上述矛盾，本章在 ZPI 索引的基础上，设计了一种基于 Z 阶曲线结构特性的最大块分割优化方案。该算法立足于 Z 阶曲线递归分割的空间特性，以查询区域的最小包含块（MCB）作为切入点，采用基于分阶中心的四分递归切割方式，将目标区域拆分为若干个尽可能大的完整 Z 阶块。相较于传统的网格

划分，最大块分割策略倾向于保留完全落在查询窗口内的高阶标准块，只有在无法完整覆盖时才进行必要的递归细化。该策略具有三个显著优势：

- (1) 通过优先保留完整标准块，最大限度减少无效点访问；
- (2) 利用 Z 阶曲线天然的遍历顺序，使生成的 Z 值区间自然有序，避免额外排序开销；
- (3) 支持在递归过程中进行区间动态合并，从而进一步降低在 PGM 索引中的查询次数。

本章将详细介绍基于 Z 阶曲线的最大块分割算法设计思想、递归分割规则及其在 ZPI 索引上的查询执行流程，并对算法复杂度进行系统分析。该方法为第五章边框扩展优化算法提供了基础，是提升 ZPI 范围查询效率的关键技术之一。

4.2 最大块分割算法设计动机

4.2.1 ZM 索引的问题分析

在执行空间范围查询时，最直观的方法是使用查询区域的左上角点和右下角点对应的 Z 值作为查询范围。然而，该范围内可能包含大量的无效的点（对应坐标在查询区域 R 之外的点），直接将其在 ZPI 中查询将导致大的查询开销和后续过滤开销。

定义 4.1 无效点：给定一个空间数据集 D 和一个查询 R ，若 D 中一个点 C 满足 $Z_Encode(R.C_0) \leq Z_Encode(C) \leq Z_Encode(R.C_3)$ 和 $C \notin R$ 的，则 C 即为一个 R 的 Z 阶曲线无效点。后文简称无效点。

问题量化：设查询区域 R 的边长为 $a \times b$ ，ZM 索引对应的 Z 值区间长度为 $Z_Encode(R.C_3) - Z_Encode(R.C_0) + 1$ ，无效点率可表示为：

$$\text{无效点率} = (\text{区间长度} - a \times b) / \text{区间长度}$$

示例 4.1：如图 4.1 所示，查询区域 R 对应的 Z 值区间[49,205]长度为 157，而 R 内实际包含的网格单元数为 $7 \times 7 = 49$ （对应 Z 值：49,51-55,57,59-63,96-111,145,147-151,153,156-157,192-201,204-205，共 49 个有效 Z 值），无效点比例高达 $(157-49)/157 \approx 68.8\%$ ，对应图中灰色区域）。

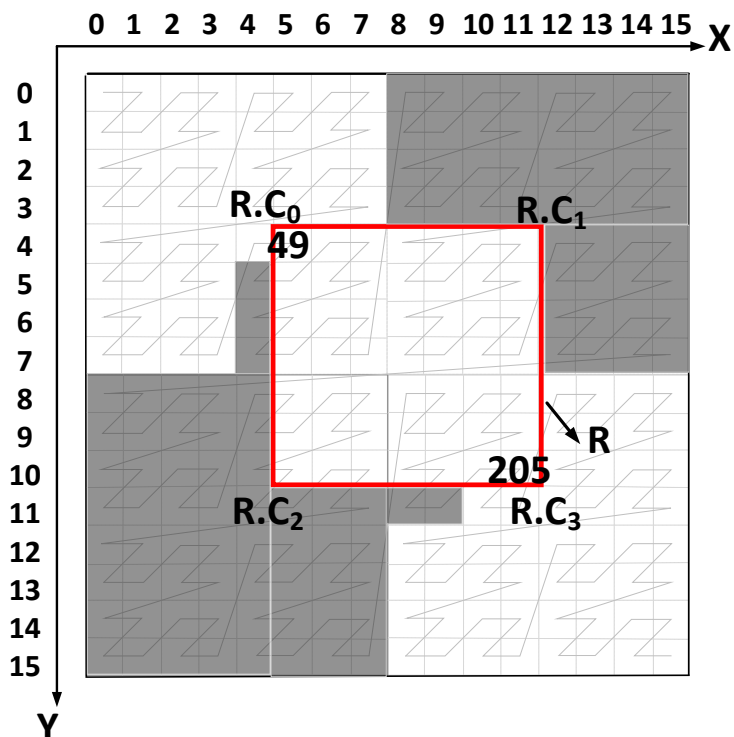


图 4.1 未分割查询区域图

4.2.2 普通分割策略的局限性

解决无效点问题的一种直观的思路是对查询区域进行分解。

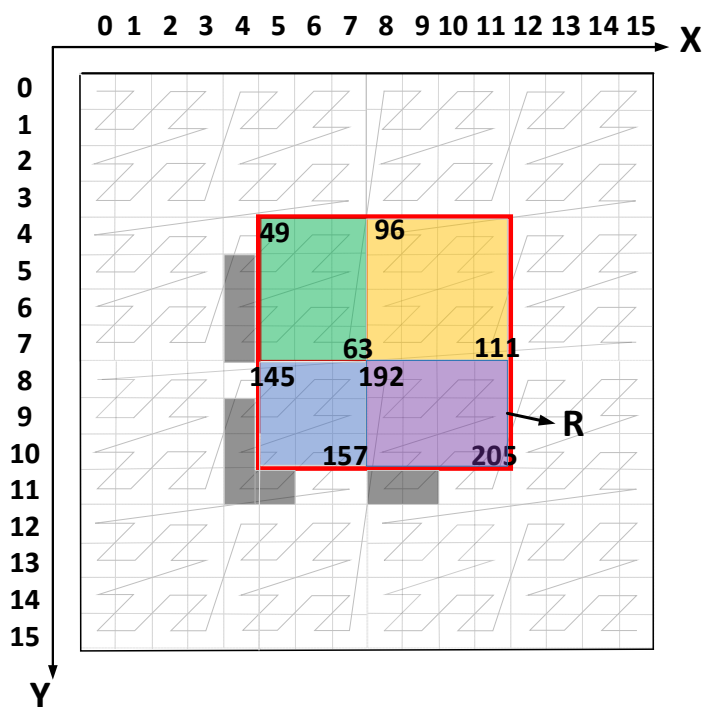


图 4.2 分割查询区域图

示例 4.2: 若将查询区域分解为四个子区域, 如图 4.2 所示, 对这四个子区域分别进行查询, 则可将上述范围缩小为 49-63(绿色区域), 96-111(黄色区域), 145-157(蓝色区域), 192-205(紫色区域) 四个较小的范围, 可有效排除大量不在查询区域内的无效点, 无效点比例为 $(58-49)/58 \approx 15.5\%$ 。

然而, 为了得到精确的查询区域范围, 必须分解到最细粒度(将 R 分解为边长为 1 的网格), 这样会产生大量细小分块。设 R 的边长为 $a \times b$, 最细粒度分解将产生 $a \times b$ 个分块。当查询范围较大时(如 $a=b=100$), 分块数量可达 10000 个, 导致在 PGM 索引中执行 10000 次小范围查询, 反而降低整体效率。

因此, 理想的分割策略应在以下两个目标之间取得平衡:

- (1) 最小化无效点: 分割后的每个子区域应尽可能减少无效点的包含;
- (2) 最小化分块数量: 分割产生的区间数量应尽可能少, 以减少索引访问次数。

为此, 提出最大块分解策略, 将查询区域分解为尽可能大的块, 每个块都为一个完整 Z 阶区域块(不会引入无效数据), 即在精确得到查询区域对应的 Z 值区间的同时, 保证 Z 值区间数最少。

4.3 最大块分割算法详细设计

4.3.1 相关定义

为便于算法描述, 首先给出相关定义如下:

定义 4.2 标准 Z 阶区域: 一个左边框的起始横坐标为 $a \cdot 2^n$, 上边框的起始纵坐标为 $b \cdot 2^n$, 且边长为 2^n 的区域称之为标准 Z 阶区域。此处 $a, b, n \in N$, N 为自然数。

定义 4.3 R 的最小包含块: 我们定义能完全包含 R 的最小的标准 Z 阶区域为 R 的最小包含块 ($R.MCB$)。

定义 4.4 分阶中心: 我们定义 $R.MCB$ 的中心为 R 的分阶中心。注意到 R 的分阶中心不一定是 R 的几何中心, 也不一定在 R 内部。如图 4.3 所示红色框为 R , 分阶中心为蓝色点。

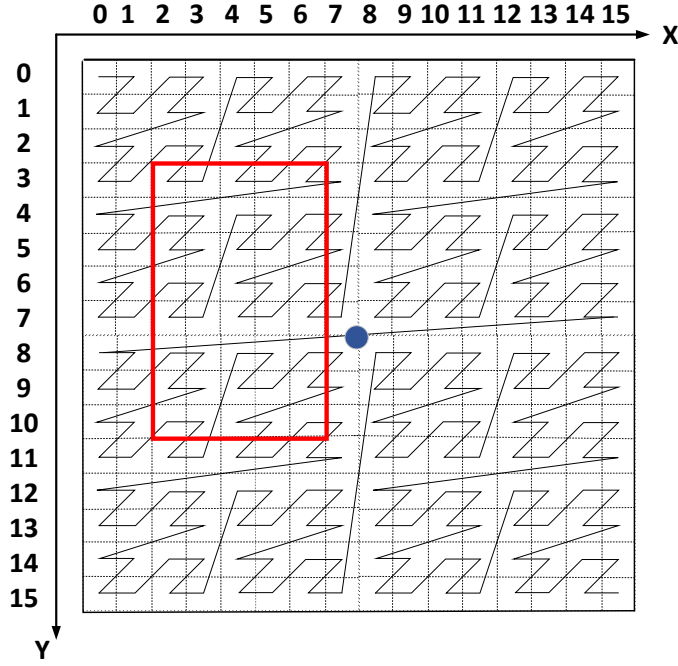


图 4.3 分阶中心示意图

性质 4.1: 对于标准 Z 阶区域, 其分阶中心将 B 划分为四个大小相等的子区域: 左上子块 B_{00} 、右上子块 B_{01} 、左下子块 B_{10} 、右下子块 B_{11} 。这四个子块的 Z 值区间按左上→右上→左下→右下的顺序递增, 且互不重叠。

4.3.2 分割策略

基于上述定义, 本节提出最大块分割的递归策略。该策略以 $R.MCB$ 为起点, 通过递归四分划分, 将 R 分解为若干完全包含于 R 内的标准 Z 阶块, 同时尽可能保留高阶块。

注意到 $R.MCB$ 是一个标准 Z 阶区域, 我们设计了高效的 QuadSplit 算法, 对其递归进行分割。分割过程中, 对任一分割后的子区域 B , 我们进行如下检测:

- (1) 若 $B \cap R = \emptyset$, 则跳过对当前子区域的检测。
- (2) 若 B 完全包含于 R 中, 则 B 中所有点均满足空间范围查询, 此时, 可终止对 B 的分割并安全地将其加入到列表 L 中。
- (3) 对其它情形, 则沿过 B 的分阶中心的水平线和垂直线对其进行 4 分割, 将其分割为 $\{B_{00}, B_{01}, B_{10}, B_{11}\}$ 4 个子区域, 然后递归对这 4 个子区域进行递归执行上述检测过程。

通过上述分割, 我们可得到对 R 的最大块分割结果。分割结束时, 我们将完全包含于 R 中的块保存在列表 L 中, 此时, 我们可得到尽可能大的完整阶的

块，从而避免简单的网格划分导致的小的分块的不足。

注意到我们将 B 分割为 $\{B_{00}, B_{01}, B_{10}, B_{11}\}$ 四个子区域之后，对四个子区域是按照 Z 阶曲线顺序访问的，故分割后得到的分块是自然地按照 Z 值大小有序排序的。得益于这一特性，为使得 QuadSplit 算法更好地服务于后续范围查询算法，我们可在分割过程中执行块的动态合并过程。具体而言，对连续插入同一个列表的两个块 B 和 B' ，若当前块 $R'.\min Z - B.\max Z = 1$ ，则可将 B' 合并到前一块 B 中。从而可降低最终获得的分块的数量，进而降低后续在 PGM 索引中查询的次数。

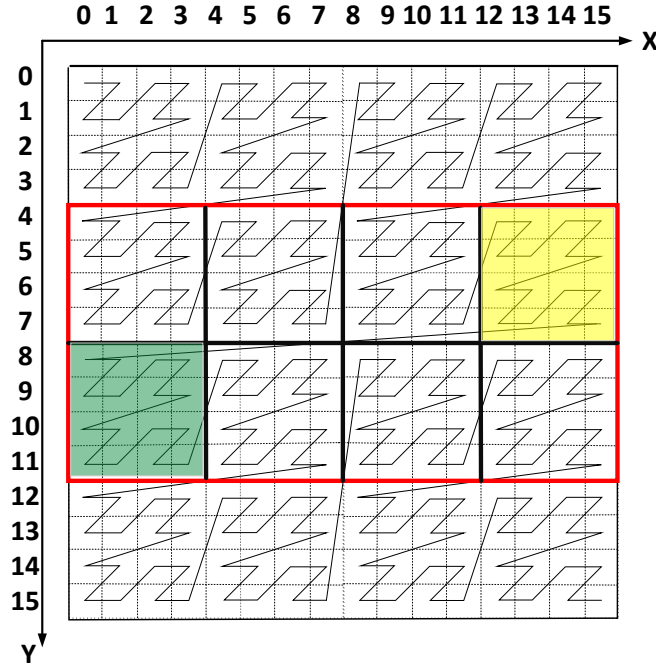


图 4.4 分块示意图

然而，直接进行块合并的效率较低，其原因如下：1) 待合并的分块大小可能不相同，导致合并后分块表示的困难，例如：一个边长为 2^m 的标准块与一个边长为 2^{m-1} 的块合并后，无法用一个简单的 Z 值范围表示。2) 并非所有待合并的分块是相邻的，例如图 4.4 中右侧的黄色块和左侧的绿色块。3) 最终我们输入 PGM 学习型索引中的是一维的 Z 值范围，而非二维的分块，最终合并后的分块仍需转换为一维的 Z 值范围。

考虑到这一点，为避免分块合并的开销，实际执行中，我们并不在 L 中存储待合并的分块，而是直接存储其一维 Z 值分段的合并，从而可有效解决上述问题。

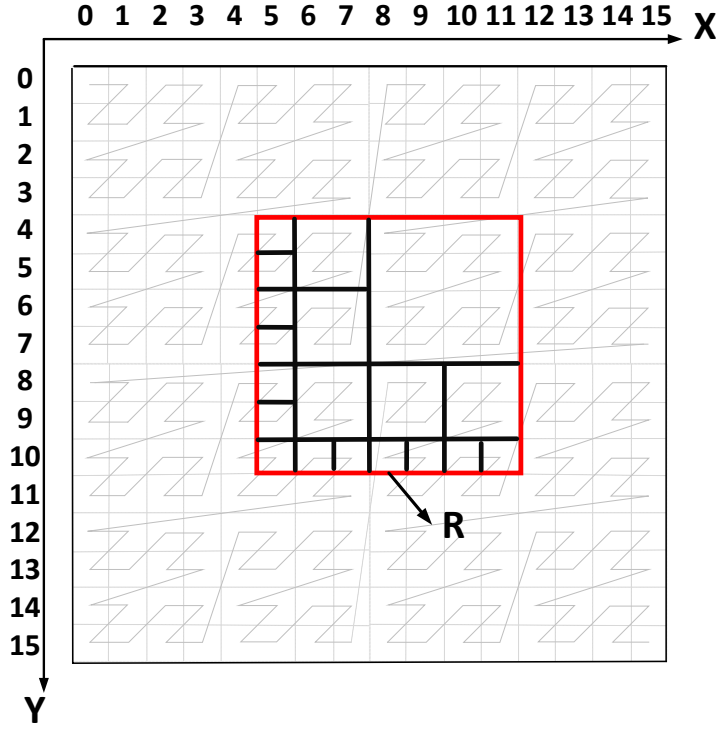


图 4.5 最大块分割结果图

通过上述分割过程，使得 R 被分割为尽可能大的块，避免了小的网格分割导致的不足，如图 4.5 所示，查询区域 R 被分割为 19 个块，每个块为一个标准 Z 阶区域，对应一个连续 Z 值段。上述分割过程具有以下优点：

- （1）分割的块为尽可能大的完整 Z 阶的块，可有效缓解直接对 R 网格分割导致的过多的块的问题。
- （2）我们沿着 Z 阶曲线前进的方向对块的分割，可在避免额外排序开销的情况下，直接合并相邻块。
- （3）分割过程中，我们得到的都是完全包含于 R 中的块，避免后续验证的代价。

4.3.3 最大块分割算法伪代码

基于上述介绍，详细的分割算法 QuadSplit 如算法 2 所示。

算法 2: QuadSplit

输入: R : 查询区域

B: Z 区域块

输出: L : 存储段的结果集

1. if $B \cap R = \emptyset$ then return
 2. else if $B \subseteq R$ then
 3. if $\min Z - \text{lastFSeg.maxZ} = 1$ then
-

```

4.      lastFSeg.maxZ=maxZ
5.      else
6.      Append [minZ,maxZ] into  L
7.      lastFSeg = [minZ,maxZ]
8.      return
9.  else
10.     { $B_{00}, B_{01}, B_{10}, B_{11}$ }  $\leftarrow$  将 B 从分阶中心四分得到四个子块
11.     QuadSplit ( R,  $B_{00}$ )
12.     QuadSplit ( R,  $B_{01}$ )
13.     QuadSplit ( R,  $B_{10}$ )
14.     QuadSplit ( R,  $B_{11}$ )

```

算法 2 采用递归四分策略对查询区域进行分解，其输入为当前待处理的 Z 阶区域块 B 与查询区域 R 。具体执行流程如下：

算法首先检查 B 与 R 的空间关系（第 1 行）。如果两者无交集（ $B \cap R = \emptyset$ ），则直接剪枝并返回，避免无效计算。

如果 B 完全包含于 R （ $B \subseteq R$ ），说明该块内所有数据点均处于查询区域内，则该块不需要进一步细分，此时算法进入会进行动态合并（第 3–8 行），通过变量 lastFSeg 记录最近加入结果列表 L 的 Z 值区间段。如果当前块的最小 Z 值 minZ 与上一区间段的最大 Z 值 lastFSeg.maxZ 连续（即 $\text{minZ} - \text{lastFSeg.maxZ} = 1$ ），则将当前块与上一个区间段合并，并且更新 lastFSeg.maxZ = maxZ。否则，将新区间[minZ, maxZ]加入到 L 并更新 lastFSeg。该机制在保持区间有序性的前提下，最大限度合并相邻区间，从而减少后续查询验证阶段在 PGM 索引中的查询次数。

如果 B 与 R 部分相交（第 9 行），算法会沿 B 的分阶中心进行四分分割，生成左上（ B_{00} ）、右上（ B_{01} ）、左下（ B_{10} ）、右下（ B_{11} ）四个子块。然后按照 Z 阶曲线的遍历顺序（左上→右上→左下→右下）递归调用 QuadSplit 处理各子块（第 11-14 行）。因为 Z 阶曲线的保序性，该递归顺序自然保证了生成的 Z 值区间按升序排列，无需额外排序操作。

通过上述递归过程，算法最终将所有完全包含于 R 的 Z 阶块以合并后的 Z 值区间形式存储于 L 中。相较于简单网格划分，该方法优先保留高阶标准块，仅在边界区域进行必要细分，在保证结果完备性的同时显著减少了分块的数量。

4.3.4 算法正确性证明

本节证明最大块分割算法的正确性，包括完备性、无冗余性和 Z 值有序性。

定理 4.1 完备性：对于查询区域 R 内任意数据点 $C \in R$ ，算法 2 必然将其

对应的 Z 值区间加入结果列表 L 。

证明：设 C 的坐标为 (x,y) ，其所在的 Z 阶网格层次结构为一条从根节点（整个空间）到叶节点（边长为 1 的网格）的路径。考虑算法对 R 的最小包含块 $R.MCB$ 的递归分割过程。由于 $C \in R$ 且 $R \subseteq R.MCB$ ， C 必然位于 $R.MCB$ 内。在递归分割过程中，当处理到包含 C 的当前块 B 时，有三种可能：

（1）若 $B \subseteq R$ ，则 B 被加入 L ，而 $C \in B$ ，故 C 的 Z 值被覆盖；

（2）若 B 与 R 部分相交但 $B \not\subseteq R$ ，算法将继续分割 B ，最终必然分割至包含 C 的最小子块 $B_{\min}$ ；

（3）由于 $C \in R$ 且 $B_{\min}$ 是包含 C 的最小网格，必有 $B_{\min} \subseteq R$ （否则 C 将不在 R 内），因此 $B_{\min}$ 最终会被加入 L 。

综上，任意 $C \in R$ 对应的 Z 值最终都会被 L 中的某个区间覆盖。

定理 4.2 无冗余性：算法 2 生成的 Z 值区间互不重叠，且每个区间对应完全包含于 R 内的标准 Z 阶块。

证明：若两个块来自同一父块的四个子块，由 Z 阶曲线性质，它们的内部互不相交， Z 值区间自然不重叠。

若两个块存在祖先-后代关系，则祖先若已加入 L ，后代不会被生成；后代若已加入 L ，则祖先必然不满足 $B \subseteq R$ （否则会在更早层次终止）。因此 L 中不会同时包含祖先和后代。

综上， L 中任意两个块的 Z 值区间无重叠。

定理 4.3 Z 值有序性：算法 2 生成的 Z 值区间列表 L 按 Z 值升序排列。

证明：我们证明递归处理顺序保证了升序性。由性质 4.1 可知，标准 Z 阶区域的四个子块按左上→右上→左下→右下的顺序，其 Z 值严格递增。算法 2 严格按照此顺序递归处理子块（第 11-14 行）。同时，完全包含块加入 L 的顺序与其被访问的顺序一致。因此， L 中的区间按 Z 值升序排列。

4.3.5 区间数量与查询框形状的关系

区间数量 N_{seg} 不仅取决于查询面积，还高度敏感于查询框的几何形状和空间位置：

长宽比影响：正方形或接近正方形的查询框更容易匹配 Z 阶曲线的递归四分结构，倾向于生成较少的大尺寸标准块；而高长宽比的细长查询框容易横跨多个高层级块的边界，导致 Z 阶曲线频繁进出查询区域，从而产生更多细碎区间。

边界对齐效应：当查询框边界恰好落在 m 阶网格线上时，分割可提前终止于 m 层，显著减少 N_{seg} 。反之，微小的位置偏移可能导致边界块被过度细分。这种因位置错位导致的区间数量波动，正是本文第五章提出边框扩展机制以稳定 N_{seg} 的主要动机。

4.4 基于最大块分割的查询算法

4.4.1 算法流程

基于前文所述的 QuadSplit 分割策略，本节给出完整的基于最大块分割的空间范围查询算法。该算法首先利用 QuadSplit 将查询区域分解为若干 Z 值区间，然后基于 PGM 索引快速定位数据，最后通过坐标映射表获取原始空间坐标。

算法 3: ZPI-RQ

输入: R : 查询区域

ZPI: ZPI 索引

输出: L : 存储段的结构集

1. $L \leftarrow \emptyset$
 2. $MCB \leftarrow \text{ComputeMCB}(R)$
 3. $\text{QuadSplit}(R, MCB, L)$
 4. $RS \leftarrow \emptyset$
 5. for each $[\min Z, \max Z]$ in L do
 6. $\text{pos_l} \leftarrow \text{ZPI.Lookup}(\min Z)$
 7. $\text{pos_r} \leftarrow \text{ZPI.Lookup}(\max Z)$
 8. $\text{start} \leftarrow \text{BinarySearch}(\text{ZPI.Az}, \min Z, \text{pos_l} - \epsilon, \text{pos_l} + \epsilon)$
 9. $\text{end} \leftarrow \text{BinarySearch}(\text{ZPI.Az}, \max Z, \text{pos_r} - \epsilon, \text{pos_r} + \epsilon)$
 10. for i from start to end do
 11. $C \leftarrow \text{ZPI.M}[i]$
 12. $RS \leftarrow RS \cup \{C\}$
 13. return RS
-

算法 3 第 1–3 行执行查询预处理。算法首先初始化区间列表 L ，然后计算查询区域 R 的最小包含块 MCB ，并以此调用算法 2 (QuadSplit) 对 R 进行递归分割。由于 QuadSplit 在分割过程中已合并相邻区间， L 中的元素即为按 Z 值升序排列的不相交区间段。

对于结果列表 L 中的每一个 Z 值区间段 $[\min Z, \max Z]$ ，算法通过以下步骤完成数据检索（第 5–12 行）：首先调用 PGM 索引的 Lookup 操作（第 6–7 行），对当前区间的 Z 值在有序数组 Az 中的存储位置进行预估。得益于 PGM 内置的误差界限 ϵ ，真实位置必然落在预测位置附近的 $[\text{pos}-\epsilon, \text{pos}+\epsilon]$ 邻域内。基于这

一确定性范围，算法无需扫描整个数组，仅需在第 8–9 行对该邻域执行二分查找，即可快速确定区间起始和结束索引。第 10–12 行遍历确定的索引范围，通过坐标映射表 A_i 直接获取原始空间坐标。由于 L 中的区间均来自完全包含于 R 的标准 Z 阶块，因此这些坐标无需额外验证即可直接加入结果集，这是与第 4 章扩展算法的关键区别。

4.4.2 与现有方法的对比分析

为更好地理解 ZPI-RQ 算法的优势与特点，本节将其与几种典型的查询方法进行理论对比分析。

1. 与 ZM 索引的对比

ZM 索引是最简单的基于 Z 阶曲线的查询方法：直接使用查询区域左上角点和右下角点的 Z 值构成单一查询区间 $[Z_encode(R.C_0), Z_encode(R.C_3)]$ 。这种方法的优点是实现简单、只需要一次索引查询，但缺点是区间内包含大量无效点。

ZPI-RQ 采用最大块分割策略，将查询区域分解为多个完全包含于 R 内的标准 Z 阶块，每个块对应一个 Z 值区间。虽然区间数量增加，但每个区间内不包含无效点，所有数据点均可直接加入结果集，无需验证。

两种方法的性能对比取决于无效点比例。当查询区域形状复杂或边界与 Z 阶曲线走向不匹配时，ZM 索引的无效点比例可达 90%，此时 ZPI-RQ 通过减少无效点访问获得的性能提升能够抵消多区间查询的额外开销。

2. 与固定网格分割的对比

固定网格分割方法将查询区域均匀划分为 $k \times k$ 个网格，对每个网格独立执行范围查询。这种方法简单直观，但存在两个主要问题：

- (1) 网格大小固定，无法自适应查询区域的形状和位置；
- (2) 无论网格是否完全包含于 R 内，均需独立查询和验证。

ZPI-RQ 的自适应分割策略具有以下优势：

(1) 优先保留高阶块：当某个区域完全包含于 R 内且尺寸较大时，直接作为一个整体查询，避免细粒度分割

(2) 区间数量更少：设查询区域边长为 a ，固定网格分割产生的区间数为 k^2 ；ZPI-RQ 的区间数量约为 $O(a)$ 量级（平均情况），远小于 k^2

(3) 自然有序：分割过程遵循 Z 阶曲线遍历顺序，生成的区间自动按 Z 值升序排列，无需额外排序

3. 与 R-tree 的对比

R-tree 作为传统空间索引的代表，通过树形结构组织空间数据，范围查询时从根节点递归遍历所有与查询区域相交的节点。

ZPI-RQ 与 R-tree 的主要区别在于：

(1) 访问路径：R-tree 可能需访问多个分支路径，且路径长度随树高增长；ZPI-RQ 在一维 PGM 索引上定位，访问路径确定且层次少

(2) 缓存局部性：ZPI-RQ 的 Z 值数组连续存储，具有良好的缓存局部性，R-tree 的节点分散存储，缓存命中率较低

(3) 构建复杂度：R-tree 需要复杂的节点分裂和重平衡操作，ZPI 仅需一次排序和线性扫描构建 PGM 模型

4.4.3 复杂度分析

算法时间复杂度分析：ZPI-RQ 算法中，QuadSplit 是算法中复杂度最高的部分，其最坏情况下需要将 $R.MCB$ 从 $R.o$ 阶递归分解到 0 阶，最多产生 $4^0 + 4^1 + \dots + 4^o = \frac{4^{o+1} - 1}{3}$ 个分区，每个分区的比较判断代价为 $c1$ ，故分割需要的时间 $\frac{4^{o+1} - 1}{3} * c1$ 。最坏情况下，我们需对每个分区在 PGM 中执行查询操作，计每次查询代价为 $c2$ ，查询的代价为 $\frac{4^{o+1} - 1}{3} * c2$ 。最坏情况下，若 R 的边长分别为 a 和 b ，边框上我们最多有 $\frac{a+b}{2^{m-1}}$ 个标准 m 阶块，均匀分布时，每块中的位置点的数量可近似估计为 $2^{2(m-n)} * |D|$ 。故验证的代价可近似为 $\frac{a+b}{2^{m-1}} * 2^{2(m-n)} * |D| * c3$ ， $c3$ 为单个点验证需要的时间，最终的时间复杂度可表示为 $\frac{4^{o+1} - 1}{3} (c1 + c2) + (a+b) * 2^{m+1-n} * |D| * c3$ 。

空间复杂度：算法需要的空间开销主要为存储 $\frac{4^{o+1} - 1}{3}$ 个分段的空间开销，对每个分块，我们存储起始和结束的 2 个 ZCode。因此，算法需要 $\frac{4^{o+1} - 1}{3} * 2 * \text{Sizeof}(\text{ZCode})$ 的空间开销。

4.5 实验分析

4.5.1 实验准备

本文所有的实验均在一台计算机上进行，该计算机 CPU 为 Intel Core Ultra

5-125H, 频率为 4.5GHz, 14 核 18 线程, 内存为 16GB, 能够提供稳定的计算环境。实验环境基于 Ubuntu 24.04 操作系统, 采用 C++ 编程语言实现, 代码使用 GCC 13.3.0 编译器编译。

本节参数设置及参数设置与 3.5 节保持一致, 此处不再赘述。

本文从查询率 r (查询范围占整个数据空间的比例), 查询区域扩展阶数 m 等角度评估这些参数对查询性能的影响。对后面的每个实验设置, 我们为每个 r 和 m 的组合, 各生成 1000 个随机位置、随机大小的查询框, 并报告查询的总时间。

4.5.2 r 对查询性能的影响

为评估 r 对查询性能的影响, 我们设置 r 分别为 0.001、0.01、0.05、0.1、0.2。为每个 r , 我们各生成 1000 个随机位置的查询。不同数据集下 ZPI-RQ 执行结果如图 4.6 所示。

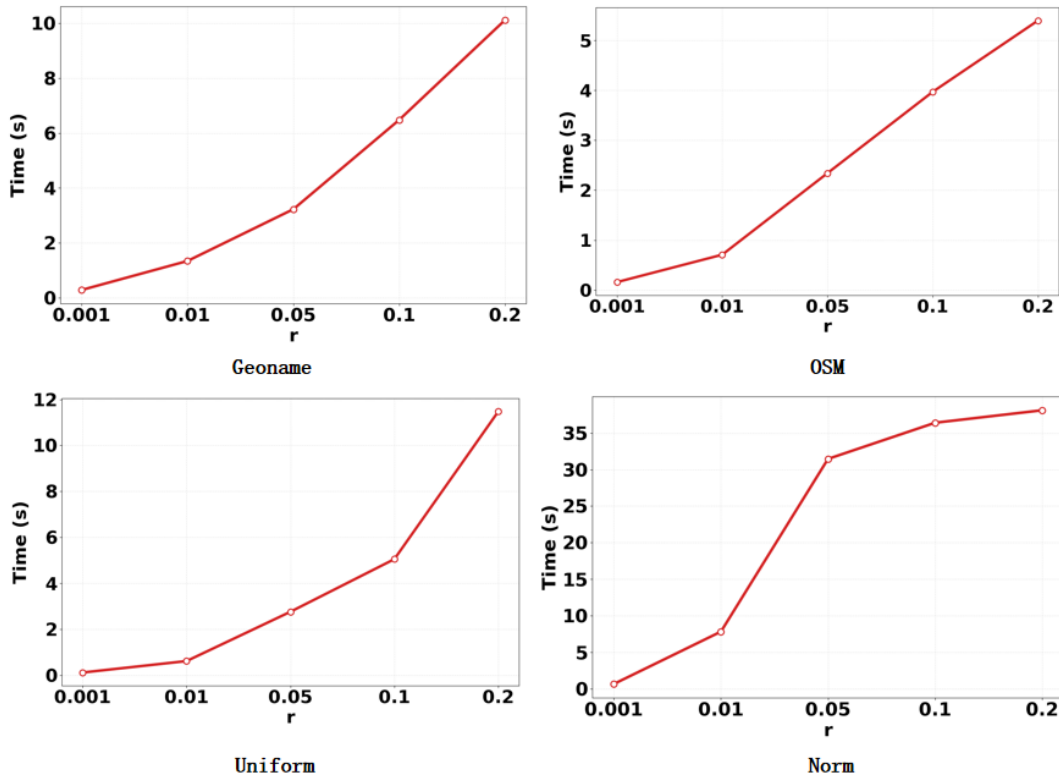


图 4.6 r 对查询性能的影响

由图 4.6 可见, 查询率 r 对查询性能有重要的影响。随着 r 的增大, ZPI-RQ 索引在四个数据集上的查询时间均呈现增加的趋势。这一现象在所有测试数据集上都得到了验证, 表明查询时间与查询范围的大小呈正相关关系。Uniform

数据集性能最好，因为数据分布均匀，分割效果稳定，Geoname 和 OSM 真实数据集性能稍差，因数据分布不均导致分割效率略低。

查询时间与查询范围的大小呈正相关关系，这是因为随着 r 的增加，查询范围覆盖的空间区域变大，落入查询框内的数据点数量也随之增加，同时候选数据数量也同步扩大，从而导致索引访问次数、数据扫描次数以及后续验证开销均随着增加。

4.5.3 和其他算法对比

为评估 ZPI-RQ 算法效率，本节将其与 R*-tree、ZM 索引、MLI 索引、LISA 索引、UG 索引、EDG 索引、Flood 索引、SPRIG 索引对比，为公平对比，考虑到 ZM、MLI、LISA、Flood 均采用了二维降为一维的策略，沿用文献^[63]介绍，我们统一采用 PGM 作为其一维底层索引结构的实现。我们固定 $m=6$ ，设置 r 从 0.001 到 0.2 变化。

对本文和 ZM、MLI、LISA、Flood 等基于学习型索引的算法，我们统一固定误差边界为 64。各数据集下，算法对比结果如图 4.7 所示。

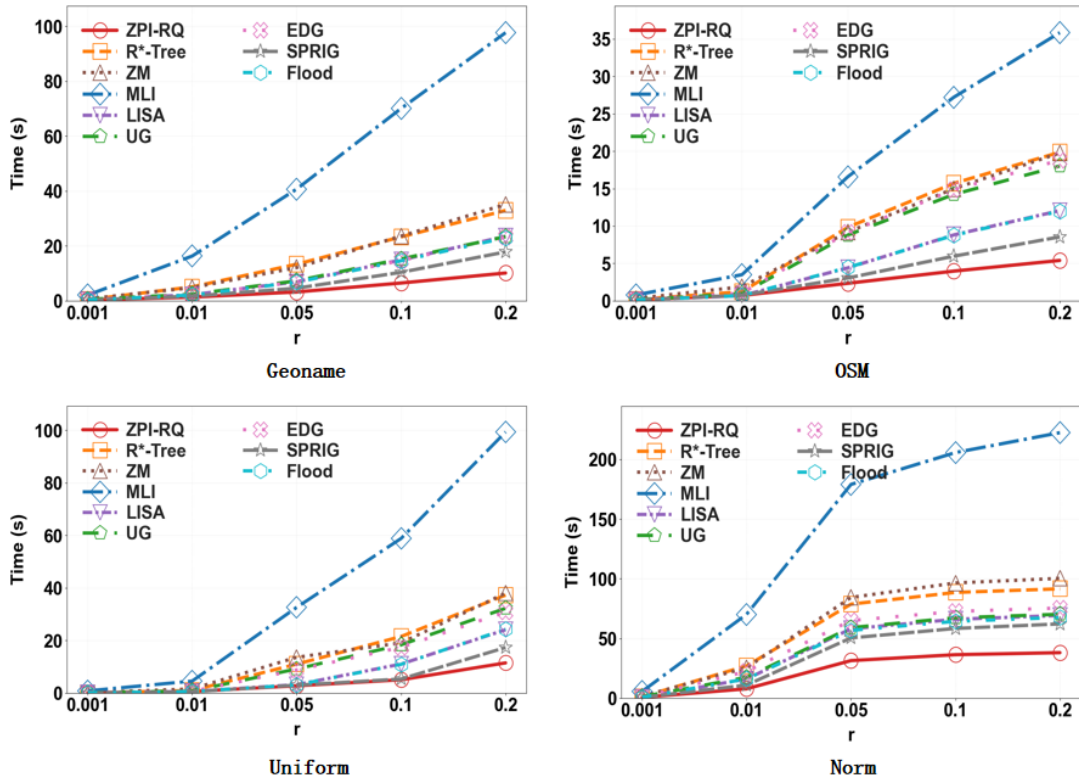


图 4.7 ZPI-RQ 与其他算法查询效率对比

ZPI-RQ 在所有测试数据集和查询率下均展现出最优或接近最优的查询性

能。特别在中等及较高查询率场景下，ZPI-RQ 的查询耗时显著低于多数对比算法，充分验证了其通过最大块分割策略有效减少无效点访问并结合 PGM 学习型索引实现快速定位的设计优势。

与 R*-tree 相比：ZPI-RQ 在绝大多数情况下优于 R*-Tree，尤其是在查询率较高时（如 0.1、0.2），最高可达 4.2 倍。例如在 Geoname 数据集上，查询率 0.1 时 ZPI-RQ 比 R*-Tree 快约 3.8 倍。这是因为 ZPI-RQ 采用了学习型索引能够快速定位数据所在区域，避免了 R*-Tree 的递归搜索开销。

与 ZM 索引相比：ZPI-RQ 在四个数据集上均显著优于 ZM 索引，特别是在高查询率下（如 0.2）性能提升平均 2–3 倍，最高 3.9 倍。这是因为本文提出的查询域分解机制有效减少了 Z 阶曲线引入的无效点访问，显著降低了后续过滤开销。

与 MLI 索引相比：ZPI-RQ 在所有查询率和数据集上均大幅领先 MLI。以 Normal 数据集为例，查询率 0.2 时 ZPI-RQ 仅为 MLI 的 17%。这是因为 MLI 基于距离降维的方法在高维或大规模数据集中计算开销较大。

与 LISA、Flood、SPRIG 等学习型索引相比：ZPI-RQ 在多数查询场景下表现更优，尤其在查询率较高时优势明显，这表明 ZPI-RQ 在降维策略与查询分解机制的协同设计上更具优势。

与 UG、EDG 相比：在极低查询率下（ $r=0.001$ ），ZPI-RQ 在多数数据集上的查询时间略高于或接近 UG 和 EDG，这主要因为查询区域极小，网格索引通过哈希直接定位的优势得以发挥。然而随着查询率增大至 $r \geq 0.01$ 时，ZPI-RQ 凭借其查询域分解机制有效减少无效点访问，性能迅速反超并保持显著优势。例如在查询率 0.2 时，ZPI-RQ 相比 UG 和 EDG 快约 1.1–1.6 倍，体现了其在处理中大范围查询时的优越性。

4.6 本章小节

针对空间范围查询里 Z 阶曲线区间膨胀这一情况，本章给出一种基于最大块分割的优化查询方案，该方案以 ZPI 索引作为基础，首先借助 Z 阶曲线把多维空间数据线性化成一维 Z 值序列，并利用 PGM 学习型索引完成快速定位；在此之上，通过递归划分查询区域，提取出完整标准 Z 阶块，有效缩小查询区间范围，从根本上降低无效点的访问量。实验数据表明，跟传统连续 Z 值区间方法做比较，最大块分割策略在中大范围查询场景下呈现稳定的性能改善，特

别在真实数据集上优势更加明显，这一改进减少后续坐标验证所需开销，证实该策略在典型应用场景具备有效性。但是，当查询区域很小或边界形态很复杂时，该算法或许会产生较多零散区间，引起索引访问次数上升，这一局限性给后续研究指出了改进路径。

整体来看，本章提出的方法在确保查询结果完备性条件下，明显提升多数场景下的空间查询效率，为下一章算法的继续优化打下理论基础。

5 基于 Z 阶曲线边框扩展的最大块分割范围查询算法

5.1 引言

第四章的最大块分割手段在多数情况下能够有效加快查询性能，不过它带来的边界处理方式却造成了一个新的效率上的冲突。为了确保查询结果能够做到完全准确，这个算法会把那些跟查询区域有交集的边界块不断进行递归切割，一直切到最精细的级别也就是边长变成 1 的网格，这种非常细致的划分方式虽然能够保证不会出现遗漏，却会带来大量分散的微小 Z 值区间。当这些区间被交给 PGM 索引去执行查询任务时，原本连续的范围检索就会被拆开成很多次碎片化的查询动作，这样一来索引访问的次数就会明显增多，整体的响应时间不但没有减少反而有所上升，怎样解决这个冲突，就变成了本章优化的一个着手点。

5.1.1 问题分析

定义 5.1 区间碎片化：在最大块分割过程中，由于查询区域边界与 Z 阶曲线网格线不对齐，这样会使边界区域被分解为大量细小、不连续的 Z 值区间，我们把这种现象称为区间碎片化。

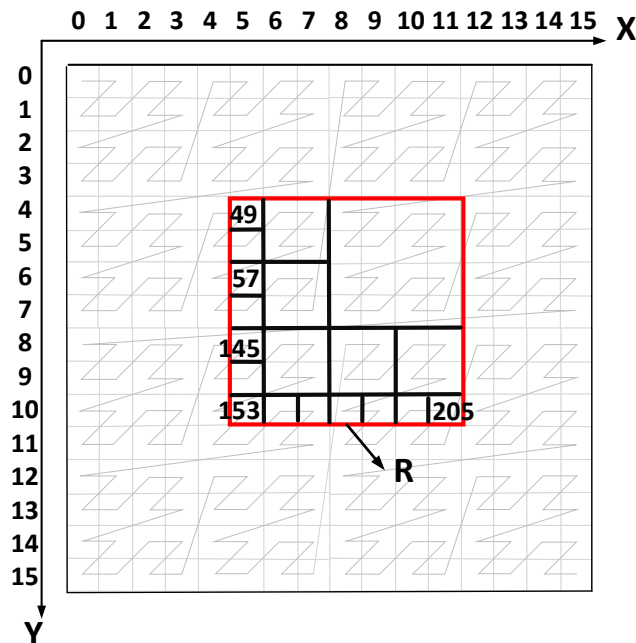


图 5.1 区间碎片化示意图

示例 5.1：如图 5.1 所示，对于一个边界复杂的查询区域 R ，采用第四章的最大块分割策略后，左侧边框将对应 49、57、145、153 等孤立的 Z 值片段。

每个片段都需要在底层 PGM 索引中单独执行一次范围查询，当片段数量增多时，索引访问次数随之增加，查询时间反而受到影响。

5.1.2 研究动机

区间碎片化问题的根源在于：Z 阶曲线在空间中的遍历路径是确定的，而查询区域的边界是任意的。当查询区域边界与 Z 阶曲线的网格线不重合时，曲线会在边界处频繁进出，导致大量细碎区间。

若查询区域的边界恰好对齐到某阶 Z 阶网格线，则整个查询区域可表示为若干个完整标准 Z 阶块的并集，此时不会产生碎片化区间。

基于这一观察，本章提出一种边框扩展机制：在正式分割之前，先将查询区域的边界适当向外调整，使其对齐到某一固定阶的 Z 阶边界线上。这样可以减少曲线在边界处的频繁进出，从而降低分割后区间数量。随后再对扩展后的区域进行最大块分割，可以在控制区间数量的同时兼顾查询精度。

接下来，本章将详细介绍边框扩展的定义与实现方法，并给出改进后的查询算法流程及其复杂度分析。

5.2 查询边框扩展机制

5.2.1 查询边框扩展

为方便描述边框扩展机制，首先给出以下定义。

定义 5.1 标准 m 阶区域：一个左边框的起始横坐标为 $a \cdot 2^m$ ，上边框的起始纵坐标为 $b \cdot 2^m$ ，且边长为 2^m 的区域称之为标准 m 阶区域。此处 $a, b \in N$ ， N 为自然数。

定义 5.2 扩展 m 阶区域：一个左边框的起始横坐标为 $a \cdot 2^m$ ，上边框的起始纵坐标为 $b \cdot 2^m$ ，且边长为 $c \cdot 2^m$ 和 $d \cdot 2^m$ 的区域称之为扩展 m 阶区域。此处 $a, b, c, d \in N$ ， N 为自然数。

一个标准 m 阶区域为一个正方形区域，而一个扩展 m 阶区域为一个长方形的区域，包含一个或多个标准 m 阶区域。我们称一个扩展 m 阶区域的上下左右四个边框线为 m 阶边框线。

定理 5.1：一个标准 m 阶区域查询框不会引入无效点。

证明：设一个标准 m 阶区域 R 的定义为：其左下角坐标为 (x_0, y_0) ，边长为 2^m 。对于区域 R 内的任意一点 $C(x, y)$ ，其坐标可表示为：

$$x=x_0+\Delta x, y=y_0+\Delta y$$

其中 $\Delta x, \Delta y \in [0, 2^m - 1]$ 。

在二进制表示下, Δx 和 Δy 均可用 m 位二进制数表示, 记作:

$$\Delta x = (x_{m-1} \dots x_0)_2, \Delta y = (y_{m-1} \dots y_0)_2$$

Z 阶曲线的编码函数 $Z(x, y)$ 通过交替拼接 x 和 y 的二进制位实现, 即:

$$(x, y) = (y_{m-1} x_{m-1} \dots y_0 x_0)_2$$

由于 Δx 和 Δy 独立地取遍所有 m 位二进制组合, 因此 $Z(x, y)$ 的低 $2m$ 位将取遍从 0 到 $2^{2m}-1$ 的所有整数值, 且这些值恰好与 R 内的点一一对应。因此, 区域 R 在 Z 值空间中对应一个连续的整数区间: $[Z_{\min}, Z_{\max}] = [Z(x_0, y_0), Z(x_0+2^m-1, y_0+2^m-1)]$ 且该区间内的每一个 Z 值都唯一对应 R 中的一个点, 不会包含 R 之外的点。故在基于 Z 阶曲线进行范围查询时, 若查询区域为标准 m 阶区域, 则其对应的 Z 值区间中不包含无效点。

定义 5.3 查询区域 R 的最小 m 阶扩展: 我们将查询区域 R 的上边框线扩展到小于等于 $R.b_t$ 的最大 m 阶边框线 $R.b_t^m$ 、将左边框线扩展到小于等于 $R.b_l$ 的最大 m 阶边框线 $R.b_l^m$ 、将右边框线扩展到大于等于 $R.b_r$ 的最小 m 阶边框线 $R.b_r^m$ 、下边框线扩展到大于等于 $R.b_b$ 的最小 m 阶边框线 $R.b_b^m$ 的过程称之为 R 的最小 m 阶扩展。

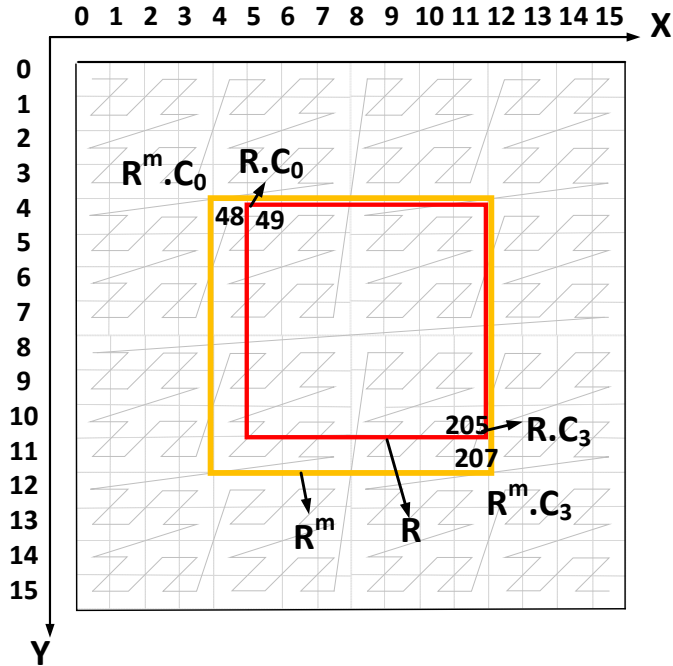


图 5.2 边框扩展示意图

通过对 R 执行最小 m 阶扩展, 我们将 R 扩展为一个扩展 m 阶区域 R^m 。 R^m 的

边框线对齐到 m 阶分割线, 因此可有效避免 Z 阶曲线频繁进出查询区域导致的大量小分段的问题。记扩展的部分 $R' = R^m - R$ 。 R^m 的左上和右下角点用 $R^m.C_0$ 和 $R^m.C_3$ 表示。注意到我们对 R 的扩展, 仅需将其角点坐标移动到新的角点, 因此扩展的实际代价较小。

示例 5.2: 如图 5.2 所示, 红色边框的查询区域 R , 对其进行最小 2 阶扩展得到黄色边框包含的区域 R^m 。扩展后, R 的左边框的 X 坐标由 5 扩展到 4, 上边框 $Y = 4$, 已经在 2 阶边框线上, 无需扩展, 右边框的 $X = 11$, 已经在 2 阶边框线上, 无需扩展, 下边框 Y 坐标由 10 扩展到 11。扩展后查询区域为 $R^m = [(4, 4), (11, 11)]$ 。

5.2.2 扩展参数 m 的选择

扩展阶数 m 是 ZPI-RQ⁺算法中最重要的参数, 直接影响查询性能的优劣。 m 的选择本质上是在两个相互制约的因素之间寻求平衡: 区间碎片化程度与扩展引入无效区域大小。

当 m 取值较小时, 扩展后的区域 R^m 与原始查询区域 R 的面积差异较小, 导致 QuadSplit⁺分割后仍需产生较多细碎区间, 碎片化改善不明显。极端情况 $m=0$ 时, 相当于不进行扩展, 退化为第四章的 ZPI-RQ 算法。

当 m 取值较大时, 边界对齐效果好, m 阶网格线较稀疏, 扩展后的边界能较好地匹配 Z 阶曲线的自然分块, QuadSplit⁺分割产生的区间数量显著减少, 索引访问次数降低, 但扩展无效区域 R' 的面积占比急剧增大, 这些区域中可能包含大量无效点, 这些无效点需要通过待验证阶段的点验证操作过滤, 增加了验证开销, 当 m 过大时, 验证开销的增长可能超过区间数量减少带来的收益, 导致整体性能下降。

在实际应用中, 查询区域的大小和形状变化多样, 很难有一个固定的 m 值适用于所有场景。但通过大量实验可以发现, 对于大多数实际查询场景, $m=6$ 是一个较为理想的选择。

5.3 基于边框扩展的最大块分割

5.3.1 改进的分割策略

尽管将 R 扩展到 R^m 之后, 可降低曲线进出查询框的次数。然而, 直接基于 R^m 的两个角点的 Z 值进行查询将导致相对 R 更大的范围, 从而降低查询效率。

示例 5.3: 如图 5.2 所示, R 对应的 Z 值范围为 49-205, 而扩展为 R^m 后对应的 Z 值范围为 48-207, 导致更多的无效点和更高的 PGM 查询开销。

对 R^m 进行最大块分割是解决上述问题的有效办法。简单来看, 我们可以对 R^m 网格分割, 将其分割为一系列 m 阶网格, 但如此分割将导致大量的较小的网格, 同样降低查询的效率。

为此, 我们设计了高效的基于分阶中心的 m 阶区域最大块递归分割的思想。首先给出相关定义如下:

定义 5.4 R^m 的最小包含块: 我们定义能完全包含 R^m 的最小的标准 Z 阶区域为 R^m 的最小包含块 ($R^m.MCB$)。

定义 5.5 R^m 的分阶中心: 我们定义 $R^m.MCB$ 的中心为 R^m 的分阶中心, $R^m.MCB$ 包含的 Z 阶曲线的阶数为 R^m 的阶数, 用 $R^m.o$ 表示。

注意到 $R^m.MCB$ 是一个标准 Z 区域, 我们设计了高效的 QuadSplit⁺算法, 对其递归进行四分分割。分割过程中, 对分割后的任一的子区域 B (为一个 $R^m.o-1$ 阶标准区域), 我们进行如下检测:

- (1) 若 $B \cap R^m = \emptyset$, 则跳过对当前子区域的检测。
- (2) 若 B 完全包含于 R 中, 则 B 中所有点均满足空间范围查询, 此时, 可终止对 B 的分割并安全地将其加入到无需验证列表 L_f 中。
- (3) 对其它情形, 若 $|B| > m$, 则沿过 B 的分阶中心的水平线和垂直线对其进行 4 分分割, 将其分割为 $\{B_{00}, B_{01}, B_{10}, B_{11}\}$ 4 个子区域 (分别对应左上、右上、左下、右下四个子区域), 然后递归对这 4 个子区域进行递归执行上述检测过程。否则若 $|B| = m$, 我们将 B 加入需要验证的列表 L_v 中。

通过对查询区域 R 进行最小 m 阶扩展并采用四叉递归分割 (QuadSplit⁺) 后, 我们得到一组候选块集合。根据块与原始查询区域 R 的关系, 可将候选块划分为两类, 从而在后续查询执行过程中采取不同处理策略:

(1) 免验证块 (free verify blocks): 若某候选块 B 完全包含于 R 内 (即 $B \subseteq R$), 则该块对应的所有数据点均满足空间范围查询条件, 可直接加入候选结果集, 无需再进行逐点验证。

(2) 待验证块 (verify blocks): 若候选块 B 与 R 相交但并不完全包含于 R (即 $B \cap R \neq \emptyset$ 且 $B \not\subseteq R$), 则该块内部可能同时包含有效点与无效点。此类块通常分布在查询区域 R 的边界附近 (包括上/下/左/右边框), 因此需要在范围查找后进一步执行逐点验证。

上述两类块分别对应 QuadSplit⁺输出的两个列表：免验证列表 L_f 与待验证列表 L_v 。特别地，待验证列表 L_v 中的块为 m 阶边框块，它们构成了候选集验证阶段的主要开销来源。免验证列表 L_f 中的块为尽可能大的完整阶的块，从而避免简单的网格划分导致的小的分块的不足。

示例 5.4：如图 5.3 所示，将最小 m 阶扩展后的查询区域进行 m 阶区域分割，得到 13 个块，其中绿色区域完全包含于 R 内，为免验证块，蓝色区域的块与 R 相交但并不完全包含于 R ，为待验证块。

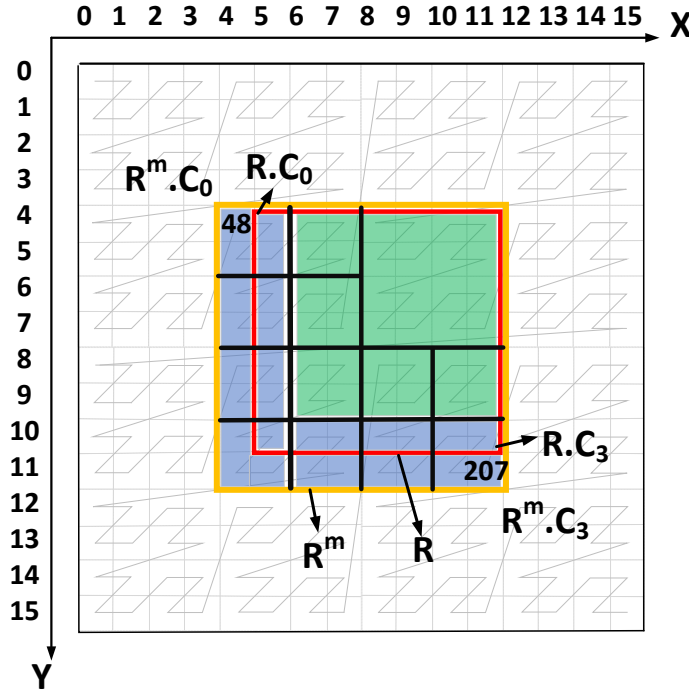


图 5.3 边框扩展示意图

通过上述分割过程，使得 R^m 被分割为尽可能大的块，避免了小的网格分割导致的不足。上述分割过程具有以下优点：

(1) 分割的块为大于等于 m 阶的块，可有效缓解直接对 R 分割导致的过小的块的问题。

(2) 沿着 Z 阶曲线前进的方向对块的分割，可在避免额外对排序开销的情况下，直接合并相邻块。

(3) 分割过程中，可以自动区分需要验证和无需验证的块，降低后续验证的代价。

5.3.2 算法正确性证明

定理 5.2（分割完备性）：对于查询区域 R 内任意数据点 $C \in R$ ，算法

QuadSplit⁺必然将其对应的 Z 值区间加入免验证列表 L_f 或待验证列表 L_v 。

证明：由定理 4.1 的证明思路， C 必然位于从 $R^m.MCB$ 到包含 C 的最小子块的路径上。在递归过程中：

若某祖先块 B 满足 $B \subseteq R$ ，则 C 被加入 L_f ，否则递归将继续直到包含 C 的最小子块 $B_{\min}$ ，由于 $C \in R$ 且 $B_{\min}$ 是包含 C 的最小网格，必有 $B_{\min}$ 与 R 相交

若 $|B_{\min}| > m$ ，算法将继续分割，最终得到阶数为 m 且包含 C 的块 B_m ，由算法规则， B_m 被加入 L_v ，因此， C 对应的 Z 值必然被 L_f 或 L_v 中的某个区间覆盖。

定理 5.3（分割无冗余性）：算法 QuadSplit⁺生成的 Z 值区间互不重叠。

证明：与定理 4.2 类似，递归分割过程中同一层次的子块互不重叠，不同层次的块要么是包含关系（继续分割）要么互不相交。当块被加入 L_f 或 L_v 后，其子块不再被处理。因此最终生成的区间互不重叠。

定理 5.4：扩展至 m 阶区域并分割后不会引入 m 阶区域之外的无效点。

证明：一个扩展 m 阶区域 R^m ，其边界在 m 阶网格线上，因此它由若干个完整的 m 阶标准区域拼成（可能是一个 m 阶块，也可能是一行/一列 m 阶块组成的矩形）。分割时，每次四叉分割的子块边界都在当前阶的网格线上，因此子块的边界始终对齐到某个等于 m 阶的网格。由定理 5.1 可知，一个完整的 m 阶标准区域不会引入无效点， R^m 自然也不会包含 R^m 以外的点。

5.3.3 QuadSplit⁺算法伪代码

QuadSplit⁺算法核心任务是计算得到免验证列表和待验证列表，以支撑后续在学习型索引 ZPI 中的查询。得益于我们在 QuadSplit⁺算法中按照 Z 阶曲线升序对块进行扫描，因此若当前访问的块的 $\min Z$ 比前一块的 $\max Z$ 大 1，则当前块的 ZCode 分段可合并到前一块的 ZCode 分段中。如此可有效降低需要在 ZPI 中执行范围查询的次数，提高算法的效率。基于上述介绍，详细的分割算法 QuadSplit⁺如算法 4 所示。

算法 4: QuadSplit⁺

输入: R : 查询区域

R^m : m 阶扩展后的查询区域

$R^m.MCB$: 查询区域 R 的最小包含块

m : 边框扩展的阶数

o : 当前 Z 区域的阶

输出: L_f : 免验证列表

L_v : 待验证列表

```

1.  if  $B \cap R^m = \emptyset$  then return
2.  else if  $B \subseteq R$  then
3.      if  $Z_B^0 - \text{lastFSeg.maxZ} = 1$  then
4.           $\text{lastFSeg.maxZ} = Z_B^3$ 
5.      else
6.          append  $[Z_B^0, Z_B^3]$  into  $L_f$ 
7.           $\text{lastFSeg} = [Z_B^0, Z_B^3]$ 
8.      return
9.  else
10.     if  $B.O = m$  then
11.         if  $Z_B^0 - \text{lastVSeg.maxZ} = 1$  then
12.              $\text{lastVSeg.maxZ} = Z_B^3$ 
13.         else
14.             append  $[Z_B^0, Z_B^3]$  into  $L_v$ 
15.              $\text{lastVSeg} = [Z_B^0, Z_B^3]$ 
16.         return
17.  else
18.       $\{B_{00}, B_{01}, B_{10}, B_{11}\} \leftarrow$  将  $B$  从分阶中心四分得到四个子块
19.      QuadSplit+( $R, R^m, B_{00}, m, o-1$ )
20.      QuadSplit+( $R, R^m, B_{01}, m, o-1$ )
21.      QuadSplit+( $R, R^m, B_{10}, m, o-1$ )
22.      QuadSplit+( $R, R^m, B_{11}, m, o-1$ )

```

QuadSplit⁺算法首先以当前待处理块 B 为输入，并结合查询区域（或扩展后的查询区域）判断 B 与查询范围的空间关系，从而决定是否继续分裂以及将结果写入哪个列表。算法第 1 行先检查 B 与查询区域是否存在交集：若 $B \cap R = \emptyset$ ，说明该块完全不可能产生结果，直接返回并结束对该分支的处理。否则，算法进入包含性判断（第 2 行）：若 $B \subseteq R$ （或在扩展版本中满足 $B \subseteq R$ 且阶数条件允许），则该块内所有点都满足范围查询条件，此时无需继续细分，算法将该块对应的 $[Z_B^0, Z_B^3]$ 区间加入免验证列表，并根据是否与上一段区间连续（即 $Z_B^0 - \text{lastVSeg.maxZ} = 1$ ）决定执行区间合并（第 3–8 行），以减少后续在一维索引中发起的范围查询次数。对于其余部分相交的情形，QuadSplit⁺会对 B 沿分阶中心进行四分，将其划分为左上、右上、左下、右下四个子块（第 18 行），并按照 Z 阶曲线的访问顺序依次递归调用 QuadSplit⁺处理四个子块（第 19–22 行）。因为我们扩展了边框，算法会在部分相交且当前块阶数已降到 m 阶时停止继续细分，此时将该块写入待验证列表，同样采用与无需验证列表一致的连续区间合并策略维护 $[Z_B^0, Z_B^3]$ 段（第 10–15 行），以便后续只对边界块做逐点验证。通过上述递归分裂与区间合并机制，QuadSplit⁺最终输出两个按 Z 值有序的区域列表：一类可直接加入结果（无需验证），另一类需要在范围查找后进一步验

证，从而在保证正确性的同时兼顾查询效率与索引访问开销。

5.4 ZPI-RQ⁺ 查询算法

5.4.1 算法设计思想

基于 QuadSplit⁺分割结果，ZPI-RQ⁺算法采用两阶段查询策略：

免验证阶段：处理 L_f 中的区间。这些区间对应的数据点全部在 R 内，可直接加入结果集，无需验证。

待验证阶段：处理 L_v 中的区间。这些区间对应的数据点可能同时包 R 内和 R 外的点，需要在获取坐标后进行点验证，仅将满足 $C \in R$ 的点加入结果集。

这种分类处理的优势在于：大部分数据点（位于内部区域的）无需验证，节省验证开销。

5.4.2 ZPI-RQ⁺算法伪代码

在 QuadSplit⁺算法基础上，我们最终实现空间范围查询算法 ZPI-RQ⁺。ZPI-RQ⁺的一个核心任务是分别对免验证列表和待验证列表里的段输入 ZPI 索引中执行查询，取出在查询区域 R 的数据。完整的 ZPI-RQ⁺算法如算法 5 所示。

算法 5: ZPI-RQ⁺

输入: Q: 查询区域

m: 边框扩展的阶数

输出: RS: 结果集

1. $RS \leftarrow \text{NULL}$
 2. $R^m \leftarrow \text{extending the border of } R \text{ to the } m\text{-th order}$
 3. $R^m.\text{MCB} \leftarrow \text{the minimum containing block of } R^m$
 4. $L_f, L_v \leftarrow \text{QuadSplit}^+(R, R^m, R^m.\text{MCB}, m, R^m.o)$
 5. for i from 0 to $|L_f|$ do
 6. for each $[\min Z, \max Z]$ in $L_f[i]$ do
 7. $\text{pos}_l \leftarrow \text{ZPI.Lookup}(\min Z)$
 8. $\text{pos}_r \leftarrow \text{ZPI.Lookup}(\max Z)$
 9. $\text{start} \leftarrow \text{BinarySearch}(\text{ZPI.Az}, \min Z, \text{pos}_l - \epsilon, \text{pos}_l + \epsilon)$
 10. $\text{end} \leftarrow \text{BinarySearch}(\text{ZPI.Az}, \max Z, \text{pos}_r - \epsilon, \text{pos}_r + \epsilon)$
 11. for each $ZCode$ in $\text{ZPI.Az}[\text{start}, \text{end}]$ do
 12. $RS \leftarrow RS \cup \{ZCode.C\}$
 13. for i from 0 to $|L_v|$ do
 14. for each $[\min Z, \max Z]$ in $L_v[i]$ do
 15. $\text{pos}_l \leftarrow \text{ZPI.Lookup}(\min Z)$
 16. $\text{pos}_r \leftarrow \text{ZPI.Lookup}(\max Z)$
 17. $\text{start} \leftarrow \text{BinarySearch}(\text{ZPI.Az}, \min Z, \text{pos}_l - \epsilon, \text{pos}_l + \epsilon)$
 18. $\text{end} \leftarrow \text{BinarySearch}(\text{ZPI.Az}, \max Z, \text{pos}_r - \epsilon, \text{pos}_r + \epsilon)$
-

```

19.      for each ZCode in ZPI. Az [start,end] do
20.          if ZCode.C in R then
21.              RS ← RS ∪ {ZCode.C}

```

算法 5 首先将 R 扩展为 R^m (第 2 行) 并计算 R^m 的 MCB (第 3 行), 然后调用 QuadSplit⁺ 算法将 R^m 分割, 获得无需验证的块列表 L_f 和需要验证的块列表 L_v (第 4 行)。算法第 5-12 行描述了对 L_f 中的块处理获得结果的过程。算法中用 minZ, maxZ 表示 L_f 中的一个段, 分别将其输入 ZPI 中, 调用 ZPI 索引的 Lookup 操作 (第 7-8 行), 对当前区间的 Z 值在有序数组 Az 中的存储位置进行预估。得益于 PGM 内置的误差界限 ϵ , 真实位置必然落在预测位置附近的 $[\text{pos}-\epsilon, \text{pos}+\epsilon]$ 邻域内。基于这一确定性范围, 算法无需扫描整个数组, 仅需在第 9-10 行对该极小邻域执行二分查找, 即可快速确定区间起始和结束索引。第 11-12 行遍历确定的索引范围, 通过坐标映射表 Ai 直接获取原始空间坐标。

对 L_v 中块的处理和 L_f 类似, 区别在于其需要对获得的 ZCode 进行验证的过程 (第 20-21 行), 得益于在 ZPI 中存储的 ZCode 对应的坐标, 实际验证中无需对 ZCode 执行额外的解码操作, 从而具有更高的效率。

5.4.3 复杂度分析

算法时间复杂度分析: ZPI-RQ⁺ 算法中, 扩展 R 为 R^m 及计算其 MCB 仅涉及两个点的扩展, 代价可忽略不计。QuadSplit⁺ 是算法中复杂度最高的部分, 其最坏情况下需要将 $R^m.MCB$ 从 $R^m.o$ 阶递归分解到 m 阶, 最多产生 $4^0 + 4^1 + \dots + 4^{o-m} = \frac{4^{o-m+1}-1}{3}$ 个分区, 每个分区的比较判断代价为 $c1$, 故分割需要的时间 $\frac{4^{o-m+1}-1}{3} * c1$ 。最坏情况下, 我们需对每个分区在 PGM 中执行查询操作, 计每次查询代价为 $c2$, 查询的代价为 $\frac{4^{o-m+1}-1}{3} * c2$ 。最坏情况下, 若 R^m 的边长分别为 a 和 b , 边框上我们最多有 $\frac{a+b}{2^{m-1}}$ 个标准 m 阶块, 均匀分布时, 每块中的位置点的数量可近似估计为 $2^{2(m-n)} * |D|$ 。故验证的代价可近似为 $\frac{a+b}{2^{m-1}} * 2^{2(m-n)} * |D| * c3$, $c3$ 为单个点验证需要的时间, 最终的时间复杂度可表示为 $\frac{4^{o-m+1}-1}{3} (c1+c2) + (a+b) * 2^{m+1-n} * |D| * c3$ 。

空间复杂度: 算法需要的空间开销主要为存储 $\frac{4^{o-m+1}-1}{3}$ 个分段的空间开销, 对每个分块, 我们存储起始和结束的 2 个 ZCode。因此, 算法需要

$\frac{4^{o-m+1}-1}{3} * 2 * \text{Sizeof}(\text{ZCode})$ 的空间开销。

5.5 实验分析

5.5.1 实验准备

本文所有的实验均在一台计算机上进行，该计算机 CPU 为 Intel Core Ultra 5-125H，频率为 4.5GHz，14 核 18 线程，内存为 16GB，能够提供稳定的计算环境。实验环境基于 Ubuntu 24.04 操作系统，采用 C++ 编程语言实现，代码使用 GCC 13.3.0 编译器编译。

本节参数设置及参数设置与 3.5 节保持一致，此处不再赘述，但在原有的基础上增加了查询域扩展阶数 m 的评估。

5.5.2 m 对查询性能的影响

为评估 m 对查询性能的影响，我们设置 m 分别为 0、2、4、6、8。为每个 m ，我们各生成 1000 个随机位置的查询。不同数据集下 ZPI-RQ⁺ 执行结果如图 5.4 所示。

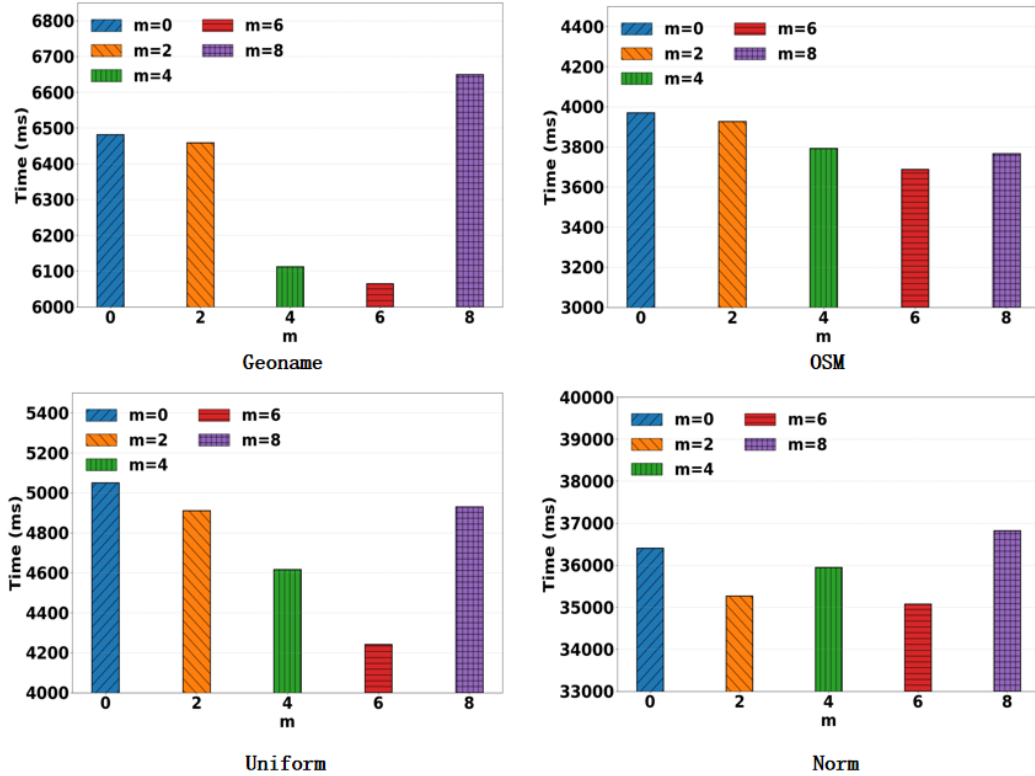


图 5.4 m 对查询性能的影响

由图 5.4 可见，扩展粒度 m 对查询性能有重要的影响。随着 m 的增大，各

数据集上的查询时间均呈现先降低后增加的趋势。这是因为，随着 m 的增加，分割后得到的分块数逐步降低，避免了大量过小的分段，从而提升了算法的效率。然而随着 m 的进一步增大， R^m 中包含的无效点的数量随着增加，从而导致算法效率降低。

总体而言，在多个数据集上，当 $m=6$ 时，算法达到或接近最优性能，其相对 $m=0$ 时（对应未扩展的情形），算法性能提升在 4.6% 到 9.8% 之间，故除特别说明外，本文设置 $m=6$ 为默认扩展的阶数。

5.5.3 r 对查询性能的影响

为评估 r 对查询性能的影响，我们设置 r 分别为 0.001、0.01、0.05、0.1、0.2。为每个 r ，我们各生成 1000 个随机位置的查询。不同数据集下 ZPI-RQ⁺ 执行结果如图 5.5 所示。

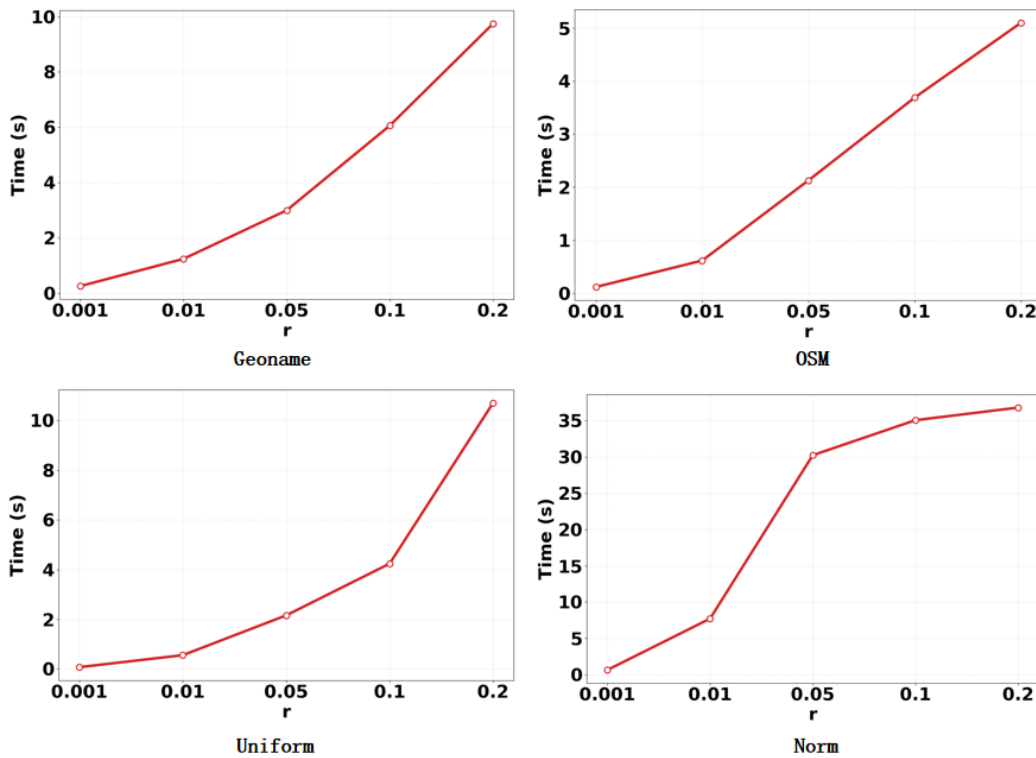


图 5.5 r 对查询性能的影响

由图 5.5 可见，查询率 r 对查询性能有重要的影响。随着 r 的增大，各数据集上的查询时间均呈现增加的趋势。这一现象在所有测试数据集上都得到了验证，表明查询时间与查询范围的大小（即查询率 r ）呈正相关关系。

这是因为，随着 r 的增加，查询范围覆盖的空间区域变大，导致需要检索

和处理的数据点数量也随之增多，从而增加了查询的总体开销。这一现象在所有测试数据集上都得到了验证，表明查询时间与查询范围的大小（即查询率 r ）呈正相关关系。

5.5.4 空间填充曲线对查询性能的影响

空间填充曲线的选择是影响降维后查询性能的关键因素之一。Z 阶曲线以其编码效率高、实现简单著称，而 Hilbert 曲线则以其更优的空间局部性保持能力闻名。为评估空间填充曲线对查询性能的影响，我们选择 Hilbert 曲线作为对照，对应的查询算法为 HPI-RQ⁺，Z 阶曲线对应的查询算法为 ZPI-RQ⁺。设置 r 分别为 0.001、0.01、0.05、0.1、0.2。为每个 r ，我们各生成 1000 个随机位置的查询。不同数据集下 HPI-RQ⁺ 和 ZPI-RQ⁺ 执行结果如图 5.6 所示。

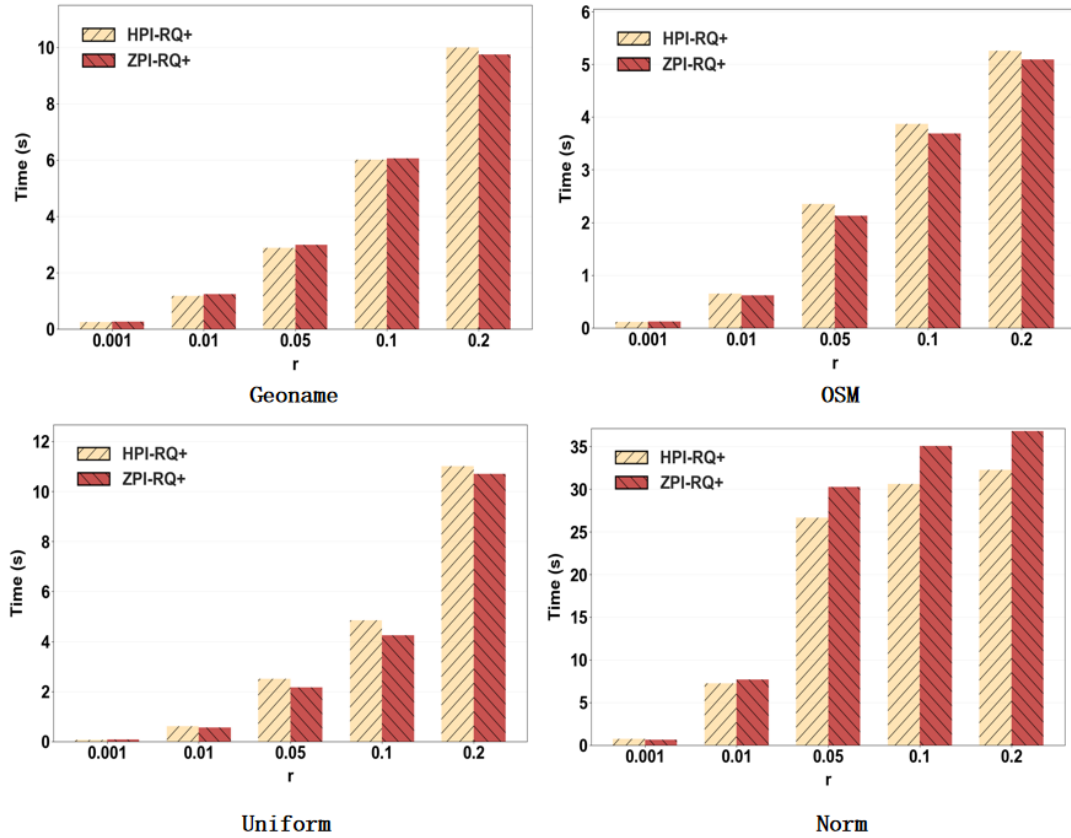


图 5.6 不同空间填充曲线查询时间对比图

由图 5.6 可知，在四组数据集的实验结果中，ZPI-RQ⁺与 HPI-RQ⁺的性能对比呈现出显著的数据集依赖性，没有一种曲线在所有场景下占绝对优势，但两者对查询性能的影响总体不大。在 Geoname、OSM、Uniform 数据集上，ZPI-RQ⁺ 均表现更优，原因是 Z 阶曲线相对更加简单，只有一种状态，而 Hilbert 曲线有

四种状态，对其进行最大块分割时，Z 阶曲线更易实现。在 Norm 数据集上 HPI-RQ⁺表现更优，原因是正态分布数据中心密集、外围稀疏，当查询率增大到 0.05 以上时，查询区域覆盖范围扩大，需要处理大量边界块。Hilbert 曲线因其更优的空间局部性保持能力，边界块内的无效点比例更低，验证开销更小。

尽管 HPI-RQ⁺的性能在部分数据集上表现更优，但 ZPI-RQ⁺在大多数情况下更优，且 Z 阶曲线编码更加简单，其核心操作是位交错，可通过查找表技术（如 VLUTg-ZE）进一步优化，将单点编码复杂度降至 $O(k/g)$ 。Hilbert 编码的优化空间有限，难以达到同等级别的编码效率。在索引构建阶段，ZPI-RQ⁺的编码时间比 HPI-RQ⁺快 3-5 倍。所以，本文最终还是选择 Z 阶曲线。

5.5.5 与 ZPI-RQ 的对比

本节直接对比 ZPI-RQ⁺与第四章提出的 ZPI-RQ 算法的查询性能，验证边框扩展策略的有效性。

固定扩展阶数 $m=6$ ，在不同查询率下对比两种算法的查询时间。为公平比较，两种算法使用相同的 ZPI 索引结构和 PGM 参数（ $\epsilon=64$ ）。

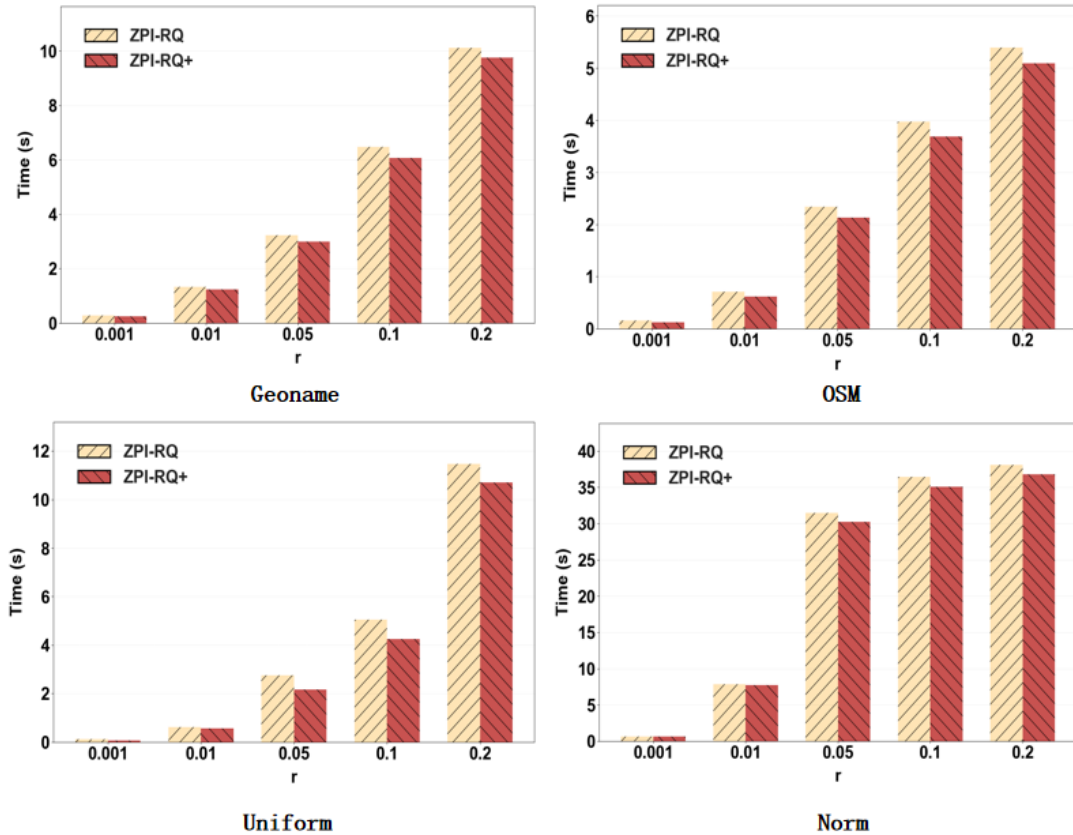


图 5.7 和 ZPI-RQ 查询时间对比图

如图 5.7 所示, $ZPI-RQ^+$ 在所有数据集和查询率下的查询时间均低于 $ZPI-RQ$ 验证了边框扩展策略的有效性。时间减少比例在 4%到 32.5%之间, 平均提升约 10.2%最大提升出现在 Uniform 数据集 $r=0.001$ 时, 查询时间从 114ms 降至 77ms, 提升 32.5%。这是因为, $ZPI-RQ^+$ 采用了边框扩展机制, 避免了查询区域被分割的太碎片化, 较少了范围查询的次数。

Uniform 数据集提升最为显著, 各查询率下时间减少比例在 6.7%-32.5%之间, $r=0.001$ 时提升 32.5%, $r=0.05$ 时提升 21.7%, $r=0.1$ 时提升 16.0%, 原因是均匀分布数据的分割边界规整, 边框扩展能有效合并边界块, 能够最大程度减少区间数量。OSM 数据集提升效果在 5.6%-24.5%之间, Geoname 数据集提升效果在 3.7%-9.6%之间。

总之, $ZPI-RQ^+$ 通过边框扩展策略, 在不同数据集和查询率下均获得了稳定的性能提升。均匀分布和真实数据集上的提升最为显著, 最高可达 32.5%。实验结果充分验证了边框扩展在减少碎片化区间、优化查询效率方面的有效性。

5.5.6 和其它算法对比

为评估 $ZPI-RQ^+$ 算法效率, 本节将其与 R^* -tree、ZM 索引、MLI 索引、LISA 索引、UG 索引、EDG 索引、Flood 索引、SPRIG 索引对比, 为公平对比, 考虑到 ZM、MLI、LISA、Flood 均采用了二维降为一维的策略, 沿用文献^[63]介绍, 我们统一采用 PGM 作为其一维底层索引结构的实现。我们固定 $m=6$, 设置 r 从 0.001 到 0.2 变化。

对本文和 ZM、MLI、LISA、Flood 等基于学习型索引的算法, 我们统一固定误差边界为 64。各数据集下, 算法对比结果如图 5.8 所示。

从图 5.8 可见, 总体上来说, 除 ZM 和 MLI 之外, 其余基于学习型索引的算法的性能普遍优于 R^* -Tree, 表明了学习型索引的优势。相对而言, 除在 $r=0.001$ 情况下, $ZPI-RQ^+$ 的性能均显著优于其它算法。 $ZPI-RQ^+$ 高效的优势在于其对边框的扩展和高效的分割。在各种查询率下, $ZPI-RQ^+$ 的查询时间高于 UG、EDG 等基于网格的算法, 其原因在于: 对较小的查询框, 网格索引的分割更简单高效, 且不会引入无效点。而 $ZPI-RQ^+$ 在扩展 m 阶之后, 扩展后增加的部分相对原有查询框的比重较大, 因此导致验证的开销较高, 这也是我们后续需要优化的重点内容之一。对随着 r 的增加, $ZPI-RQ^+$ 分割将可能产生大的无需验证的块, 因此效率逐步提升。相对于其它同样基于 PGM 实现的算法, $ZPI-RQ^+$ 性能提升在 1.5 倍到 2.4 倍之间, 表明了我们算法的优势。相对于同为

基于 Z 阶曲线的算法 ZM, ZPI-RQ⁺的性能提升最高可达到 6.3 倍, 这说明本文提出的查询框扩展及分解机制有效减少了 Z 阶曲线引入的无效点访问, 显著降低了后续过滤开销。

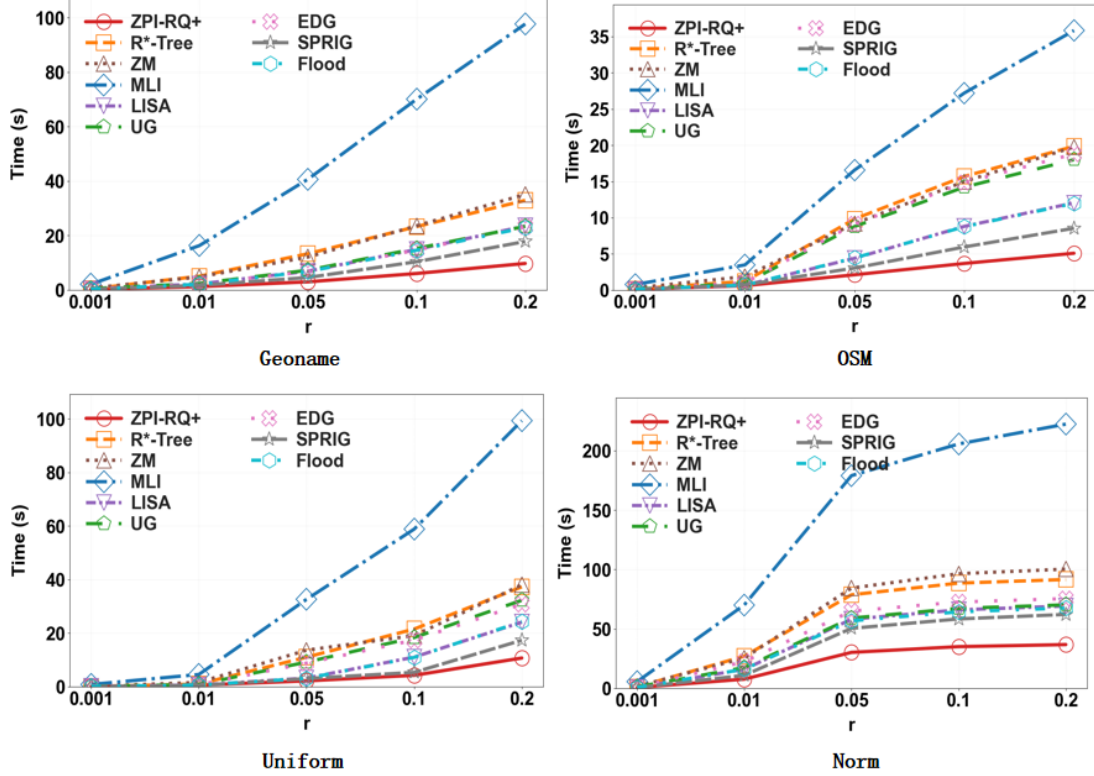


图 5.8 不同索引查询时间对比图

总体而言, 在 4 个数据集上, ZPI-RQ⁺相对最好的竞争者的性能提升最高可分别为: 1.8 倍, 1.6 倍, 1.4 倍和 1.2 倍。

5.6 本章小结

在第四章最大块分割方法于边界处理过程中出现的区间碎片化这一问题上, 本章带进边框扩展机制, 给出改进后的范围查询算法 ZPI-RQ⁺, 该算法的核心思路是把查询区域向外扩展至 m 阶边界线, 如此便能减少 Z 阶曲线在边界位置频繁进出的情况, 从而减少区间碎片化现象。

在区间处理这一环节, ZPI-RQ⁺对候选块进行分类处理, 如果完全落入查询区域的块直接归入结果集, 如果部分落入查询区域则对候选块进行坐标验证。这种差异化处理在保证查询准确性的同时, 有效削减了验证带来的开销。

理论分析同实验数据都显示,扩展阶数 m 是影响查询性能的一项关键参数,合理选择 m 值能够在区间数量与无效点访问之间取得有效平衡,这样能够优化整体查询效率。

总的来说, ZPI-RQ^+ 在第四章方法的基础上,借助边框扩展和分类处理机制,进一步增强了查询执行的效率,为后续研究的深入打下基础。

6 结论

6.1 本文工作总结

这篇文章把关注点放在 Z 阶曲线在空间范围查询里碰到的两个关键性能瓶颈问题，也就是无效点访问过多和边界区间碎片化现象，系统化地展开了对索引结构跟查询算法进行优化的研究工作。

在索引这个层面，构建了 ZPI 混合索引结构，该方案借助 Z 阶曲线把二维空间数据线性化成一维 Z 值序列，并引入 PGM 学习型索引来实现数据位置的快速预测，相较于传统的树型空间索引，该结构显著简化了索引组织方式，构建流程更为高效。在查询层面，首先针对查询区域角点 Z 值会引入大量无需数据增加过滤开销问题，提出了最大块分割策略。通过递归划分查询区域，提取尽可能大的完整的标准 Z 阶块，有效压缩了查询区间范围，减少了无效数据访问。实验表明，该方法在中大范围查询场景下显著降低了查询延迟。针对分割后边界区域产生大量小区间的衍生问题，进一步设计了边框扩展机制，形成了完整的 ZPI-RQ⁺ 查询算法。通过将查询区域对齐至固定阶网格边界，减少了分段数量；同时采用分类验证策略，区分完全包含块与边界块，在保障结果正确性的前提下控制了验证开销。实验结果显示，改进后的算法在多种数据集上均展现出稳定的性能提升。

整体来看，这篇文章围绕减少无效点访问这个核心目标，对基于 Z 阶曲线的学习型空间索引进行了系统性的改进与验证，为相关研究提供了有价值的参考。

6.2 工作展望

尽管本文在查询效率优化方面取得了阶段性成果，但研究工作仍存在若干值得深入探索的方向。

动态更新机制：当前 ZPI 索引主要面向静态数据集，面对频繁更新的应用场景需完全重建模型，带来额外开销。后续可引入支持增量更新的学习型索引结构，提升索引在动态环境下的实用性。

高维数据扩展：本文方法目前局限于二维空间数据。随着三维点云、时空数据等高维应用日益普及，如何在维度扩展的同时保持查询性能，是亟待解决的关键问题。

参数自适应调优：边框扩展中的阶数 m 目前依赖实验经验确定，在不同数据分布下可能非最优。未来可研究基于数据特征或查询模式的自适应参数选择策略，使算法能够自动匹配最优配置。

总体而言，基于 Z 阶曲线与学习型索引的空间查询方法仍有广阔的发展空间。随着数据规模的增长和应用场景的演变，相关技术将持续演进和完善。

致谢

时光飞逝，转眼间研究生三年的时光就要结束了，回首这一段时光，我从刚踏入科研之门，带着好奇与懵懂，到如今完成已在科研上取得了一点成果，完成了论文的撰写，一路走来，百感交集。这篇论文不仅是我学术探索的一份答卷，更承载了这一段时光无数的关爱与支持。

首先，我要向我的恩师贾连印教授致以最诚挚的谢意。从选题时反反复复的斟酌，到实验遇到瓶颈时的耐心点拨，再到逐字逐句的修改论文，您不仅教会我怎样从事科研，更教会了我对待事情持之以恒、一丝不苟的态度。每次看到你帮我一字一句修改的论文，都能感受到你背后要付出多少的时间和精力，我的论文能够如此圆满的完成，离不开您的指导，您严谨的治学态度始终指引着我前进，并且将会成为我以后成长路上宝贵的财富。

其次，我要感谢我的父母。二十余载的求学路，是你们用日渐佝偻的身体为我撑起了一片天空，我在你们撑起的天空无忧的成长。你们没读过多少书，却让我能够无忧无虑的读书。从最开始决定考研，到如今学业快完成，我做出的决定你们总是能无条件的支持，让我有底气去追寻自己的梦想。

最后，我要感谢我所有的同学们，是你们的陪伴让我在这一路上能够走的更加丰富多彩，从你们的身上我学到了很多在书本上学不到的东西，和你们在一起让我获得了很多快乐，这些都将永远的留在我的心底。

凡是过往，皆为序章。虽然学生生涯即将结束，但探索未知的脚步永远都不会停歇。我会带着这份感激与收获，奔赴人生的下一场山海。祝愿老师身体健康，祝愿同学们前程似锦。

参考文献

- [1] Yulei Wu, Hong - Ning Dai, Haozhe Wang, et al. A Survey of Intelligent Network Slicing Management for Industrial IoT: Integrated Approaches for Smart Transportation, Smart Energy, and Smart Factory[J]. IEEE Communications Surveys & Tutorials, 2022, 24(2): 1175-1211.
- [2] Agouris P, Aref W, Goodchild M F, et al. From GPS and virtual globes to spatial computing-2020[C]//The next transformative technology, computing community consortium (CCC) workshop proposal. 2012.
- [3] Damilola Oladimeji, Khushi Gupta, Nuri Alperen Kose, et al. Smart Transportation: An Overview of Technologies and Applications[J]. Sensors, 2023, 23(8): 3880.
- [4] Martin Breunig, Patrick Erik Bradley, Markus Jahn, et al. Geospatial Data Management Research: Progress and Future Directions[J]. ISPRS International Journal of Geo-Information, 2020, 9(2): 95.
- [5] Sun L M, Zhang S M, Ji T, et al. Survey of data management techniques powered by artificial intelligence[J]. Journal of Software, 2020, 31(3): 600-619.
- [6] Marks D, Abdelguerfi M, Ioup E, et al. A System Overview for netCDF-FastBit Integration[J]. DBKDA 2012, 2012: 154-156.
- [7] Wang P, Ravishankar C V. Secure and efficient range queries on outsourced databases using Rp-trees[C]//2013 IEEE 29th international conference on data engineering (ICDE). IEEE, 2013: 314-325.
- [8] Guttman A. R-trees: A dynamic index structure for spatial searching[C]//Proceedings of the 1984 ACM SIGMOD international conference on Management of data. 1984: 47-57.
- [9] Wu C H, Chang L P, Kuo T W. An efficient R-tree implementation over flash-memory storage systems[C]//Proceedings of the 11th ACM international symposium on Advances in geographic information systems. 2003: 17-24.
- [10] Zhu Q, Gong J, Zhang Y. An efficient 3D R-tree spatial index method for

- or virtual geographic environments[J]. ISPRS Journal of Photogrammetry and Remote Sensing, 2007, 62(3): 217-224.
- [11]Beckmann N, Kriegel H P, Schneider R, et al. The R*-tree: An efficient and robust access method for points and rectangles[C]//Proceedings of the 1990 ACM SIGMOD international conference on Management of data. 1990: 322-331.
- [12]Kraska T, Beutel A, Chi E H, et al. The case for learned index structures[C]//Proceedings of the 2018 international conference on management of data. 2018: 489-504.
- [13]Beckmann N, Seeger B. A revised R*-tree in comparison with related index structures[C]//Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data. New York: ACM, 2009: 799-812.
- [14]Sellis T, Roussopoulos N, Faloutsos C. The R+-tree: A dynamic index for multi-dimensional objects[R]. Pittsburgh: Carnegie Mellon University, 1987.
- [15]Samet H. The quadtree and related hierarchical data structures[J]. ACM Computing Surveys (CSUR), 1984, 16(2): 187-260.
- [16]Jung H R, Kim Y S, Chung Y D. QR-tree: An efficient and scalable method for evaluation of continuous range queries[J]. Information Sciences, 2014, 274: 156-176.
- [17]Finkel R A, Bentley J L. Quad trees a data structure for retrieval on composite keys[J]. Acta informatica, 1974, 4: 1-9.
- [18]Kothuri R K V, Ravada S, Abugov D. Quadtree and R-tree indexes in Oracle spatial: a comparison using GIS data[C]//Proceedings of the 2002 ACM SIGMOD international conference on Management of data. 2002: 546-557.
- [19]Islam M S, Liu C, Rahayu W, et al. Q+ tree: An efficient quad tree based data indexing for parallelizing dynamic and reverse skylines[C]//Proceedings of the 25th ACM International on Conference on Information and Knowledge Management. 2016: 1291-1300.
- [20]Markas T, Reif J. Quad tree structures for image compression applications

- [J]. Information Processing & Management, 1992, 28(6): 707-721.
- [21] Samet H. Data structures for quadtree approximation and compression[J]. Communications of the ACM, 1985, 28(9): 973-993.
- [22] Zheng B, Xu J, Lee W C, et al. Grid-partition index: a hybrid method for nearest-neighbor queries in wireless location-based services[J]. The VL DB Journal, 2006, 15(1): 21-39.
- [23] Park K. Location-based grid-index for spatial query processing[J]. Expert systems with applications, 2014, 41(4): 1294-1300.
- [24] Jaewoo Shin, Ahmed R. Mahmood, Walid G. Aref. An Investigation of Grid-enabled Tree Indexes for Spatial Query Processing[J]. 2019, 169-178.
- [25] ZHANG Li-fen, WANG Xiao-hua, HU Jing-song, SONG Wei-jia, LONG Bin. Spatial Indices Based on Grid Partition[J]. Transactions of Beijing institute of Technology, 2004, (2): 140-144.
- [26] Graefe G, Larson P A. B-tree indexes and CPU caches[C]//Proceedings 17th International Conference on Data Engineering. IEEE, 2001: 349-358.
- [27] Graefe G. B-tree indexes for high update rates[J]. ACM Sigmod Record, 2006, 35(1): 39-44.
- [28] Berger C, Birkemeyer L. Zebra: Z-order curve-based event retrieval approach to efficiently explore automotive data[C]//2023 IEEE Intelligent Vehicles Symposium (IV). IEEE, 2023: 1-7.
- [29] Wang J, Shan J. Space filling curve based point clouds index[C]//Proceedings of the 8th International Conference on GeoComputation. 2005: 551-562.
- [30] Dai H K, Su H C. Clustering Analyses of Two-Dimensional Space-Filling Curves: Hilbert and z-Order Curves[J]. SN Computer Science, 2023, 4(1): 8-1-8-19.
- [31] Chen J, Yu L, Wang W. Hilbert space filling curve based scan-order for point cloud attribute compression[J]. IEEE Transactions on Image Processing, 2022, 31: 4609-4621.
- [32] Butz A R. Alternative algorithm for Hilbert's space-filling curve[J]. IEEE Transactions on Computers, 2006, 100(4): 424-426.

- [33]Lawder J K, King P J H. Querying multi-dimensional data indexed using the Hilbert space-filling curve[J]. *ACM Sigmod Record*, 2001, 30(1): 19-24.
- [34]Ding J, Nathan V, Alizadeh M, et al. Tsunami: A learned multi-dimensional index for correlated data and skewed workloads[J]. *arXiv preprint arXiv:2006.13282*, 2020.
- [35]Galakatos A, Markovitch M, Binnig C, et al. Fiting-tree: a data-aware index structure//*Proceedings of the International Conference on Management of Data*. Amsterdam, The Netherlands, 2019: 1189-1206.
- [36]Ferragina P, Vinciguerra G. The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds[J]. *Proceedings of the VLDB Endowment*, 2020, 13(8): 1162-1175.
- [37]Ding J, Minhas U F, Yu J, et al. ALEX: an updatable adaptive learned index[C]//*Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2020: 969-984.
- [38]Wu J, Zhang Y, Chen S, et al. Updatable learned index with precise positions[J]. *arXiv preprint arXiv:2104.05520*, 2021.
- [39]Wang H, Fu X, Xu J, et al. Learned index for spatial queries[C]//*2019 20th IEEE International Conference on Mobile Data Management (MDM)*. IEEE, 2019: 569-574.
- [40]Qi J, Liu G, Jensen C S, et al. Effectively learning spatial indices[J]. *Proceedings of the VLDB Endowment*, 2020, 13(12): 2341-2354.
- [41]Davitkova A, Milchevski E, Michel S. The ML-Index: A multidimensional, learned index for point, range, and nearest-neighbor queries//*Proceedings of the 23rd International Conference on Extending Database Technology*. Copenhagen, Denmark, 2020: 407-410.
- [42]Li P, Lu H, Zheng Q, et al. LISA: A learned index structure for spatial data[C]//*Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2020: 2119-2133.
- [43]Kang - Tsung Chang. Geographic Information System[J]. *International Encyclopedia of Geography*, 2017, 1-9.

- [44] Spatial data types for database systems: finite resolution geometry for geographic information systems[M]. Berlin, Heidelberg: Springer Berlin Heidelberg, 1997.
- [45] Psyllidis A, Gao S, Hu Y, et al. Points of Interest (POI): A commentary on the state of the art, challenges, and prospects for the future[J]. Computational urban science, 2022, 2(1): 20.
- [46] Baert J. Morton encoding/decoding through bit interleaving: Implementations[J]. retrieved from, Oct, 2013, 7: 13.
- [47] Zhou Z, Sun S, Liang T, et al. Efficient OpenMP Based Z-curve Encoding and Decoding Algorithms[J]. Computers, Materials & Continua, 2025, 82(1).
- [48] Baert J. Libmorton: C++ Morton encoding/decoding library[J]. Accessed: Aug, 2018, 5: 2021.
- [49] Jia L, Xu J, Ding J, et al. Efficient Lookup Table Based Z-Order Curve Encoding and Decoding: Algorithms, Parallelization, and Applications[J]. Tsinghua Science and Technology, 2025.
- [50] Bentley J L. K-d trees for semidynamic point sets[C]//Proceedings of the sixth annual symposium on Computational geometry. 1990: 187-197.
- [51] Ram P, Sinha K. Revisiting kd-tree for nearest neighbor search[C]//Proceedings of the 25th acm sigkdd international conference on knowledge discovery & data mining. 2019: 1378-1388.
- [52] Procopiuc O, Agarwal P K, Arge L, et al. Bkd-tree: A dynamic scalable kd-tree[C]//International symposium on spatial and temporal databases. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003: 46-65.
- [53] Qin J, Zhu X, Liu Q, et al. Piecewise linear approximation in learned index structures: Theoretical and empirical analysis[EB/OL]. arXiv, 2025.
- [54] Ganguly A, Mitra D, Balakrishnan N, et al. Analysis of Left Truncated Right Censored Data with Covariates via a Flexible Model based on Piecewise Linear Approximation[J].
- [55] 李国良,周煊赫,孙佶,et al.基于机器学习的数据库技术综述[J].计算机学报, 2020, 43(11):31.
- [56] 蔡盼,张少敏,刘沛然,等. 智能数据库学习型索引研究综述[J]. 计算机学

报,2023,46(1):51-69.

- [57]Nathan V, Ding J, Alizadeh M, et al. Learning multi-dimensional indexes [C]//Proceedings of the 2020 ACM SIGMOD international conference on management of data. 2020: 985-1000.
- [58]Ferragina P, Vinciguerra G. The PGM-index: a multicriteria, compressed and learned approach to data indexing[J]. arXiv preprint arXiv:1910.06169, 2019.
- [59]Maltry M, Dittrich J. A critical analysis of recursive model indexes[J]. arXiv preprint arXiv:2106.16166, 2021.
- [60]<https://www.geonames.org>.
- [61]osm-china [n.d.]. OpenStreet Map. <https://planet.openstreetmap.org>. Accessed: 2025-04-15.
- [62]Zhang S, Ray S, Lu R, et al. SPRIG: A learned spatial index for range and kNN queries[C]//Proceedings of the 17th international symposium on spatial and temporal databases. 2021: 96-105.
- [63]Liu Q, Li M, Zeng Y, et al. How good are multi-dimensional learned indexes? An experimental survey[J]. The VLDB Journal, 2025, 34(2): 17.

附录 A 攻读硕士学位期间的学术成果

学术论文

- [1] Lianyin Jia, Rongjin Wang, Yingbin Su, Suprio Ray, Mengjuan Li, Jiaman Ding.
Efficient Query Region Expansion and Decomposition based Spatial Range
Query Algorithm. (CCF A 类会议已录用)

已受理发明专利

- [1] 贾连印, 汪荣进, 李孟娟, 丁家满, 周海河, 曾瑞, 李晓武. 基于 Z 阶曲线最大块分割的空间范围查询方法及其应用. (实质审查)

在读期间获奖情况

- [1] 2024.12 第二十一届“华为杯”中国研究生数学建模大赛国家二等奖

附录 B 攻读硕士学位期间参与的科研项目

[1]

[2]

[3]

[4]

